Data visualization with ggplot2

Day 2 - Introduction to Data Analysis with R

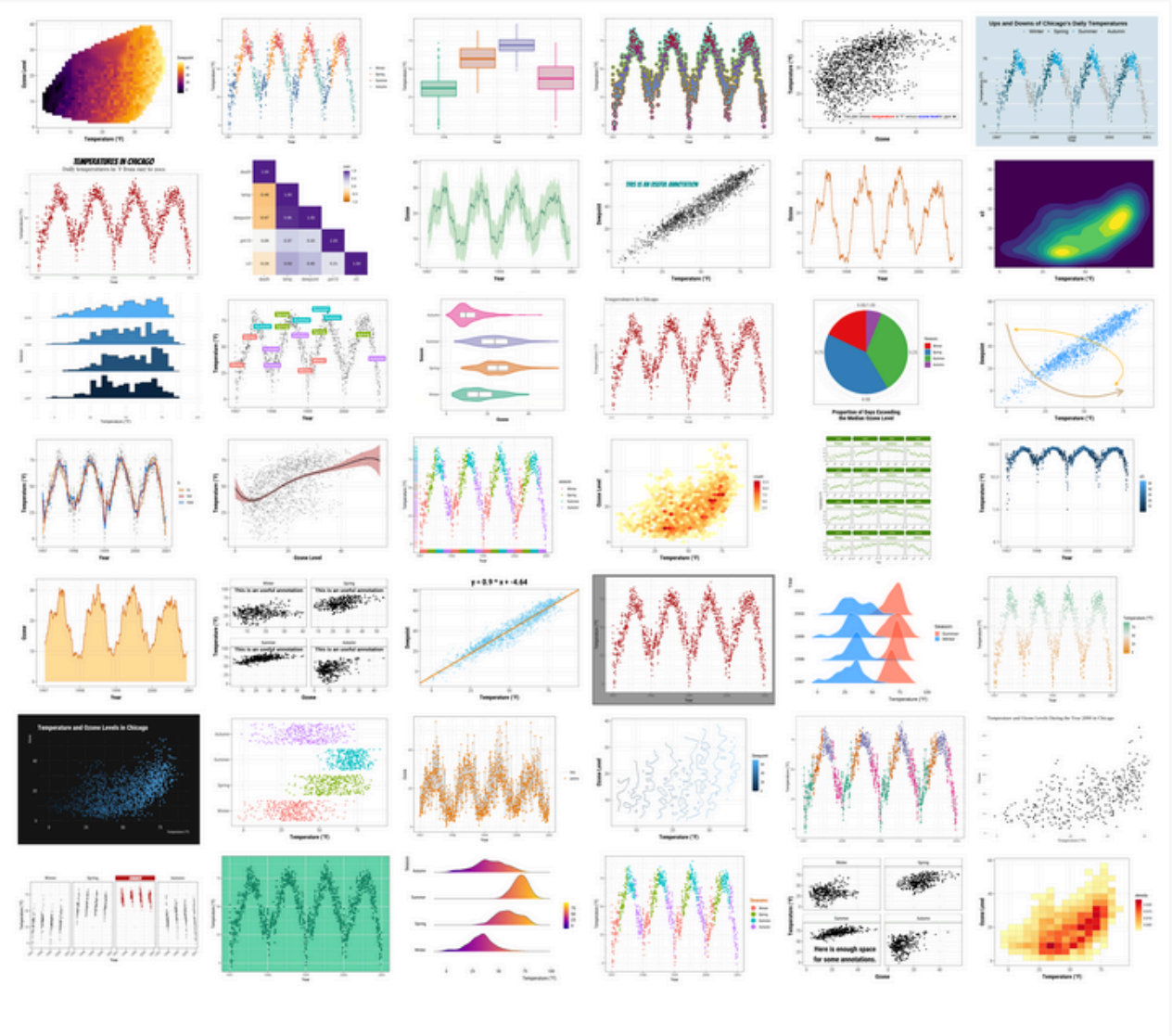
Selina Baldauf
Freie Universität Berlin - Theoretical Ecology

October 12, 2026

A ggplot showcase

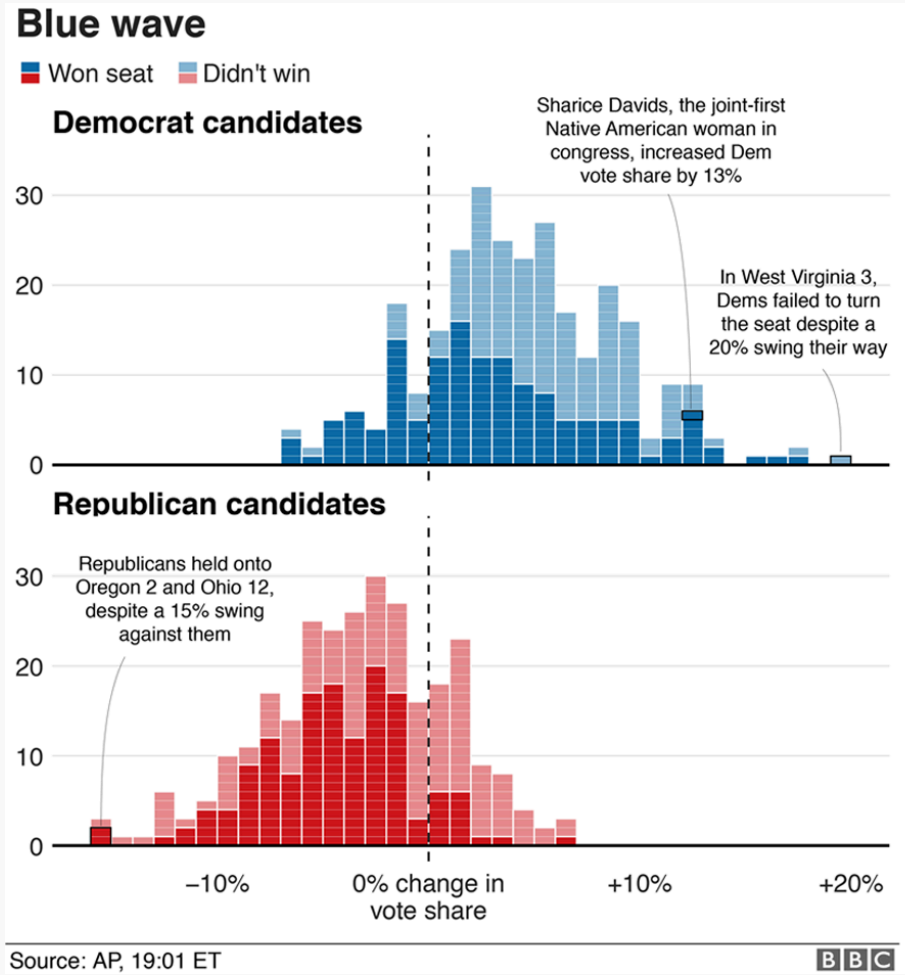
Example plots you can create with ggplot

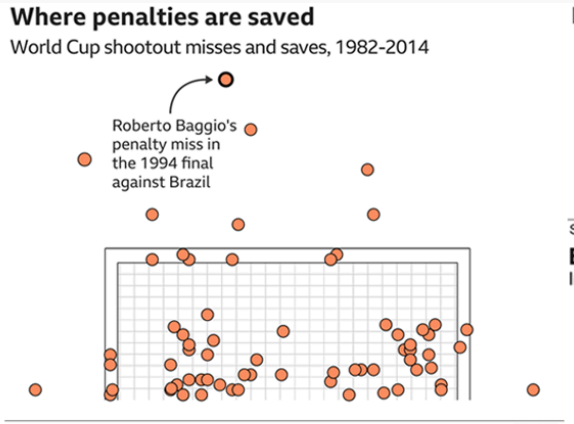
A ggplot showcase

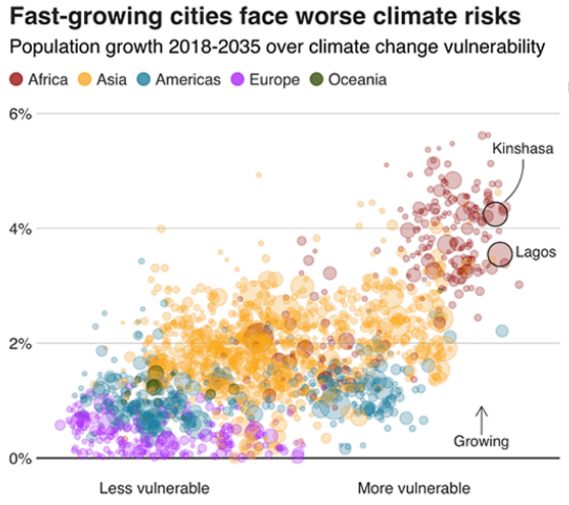


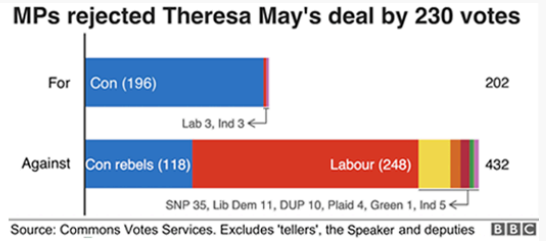
Overview of different plot possibilites created by Cédric Scherer. Check out his tutorial to learn how to make all these plots with ggplot

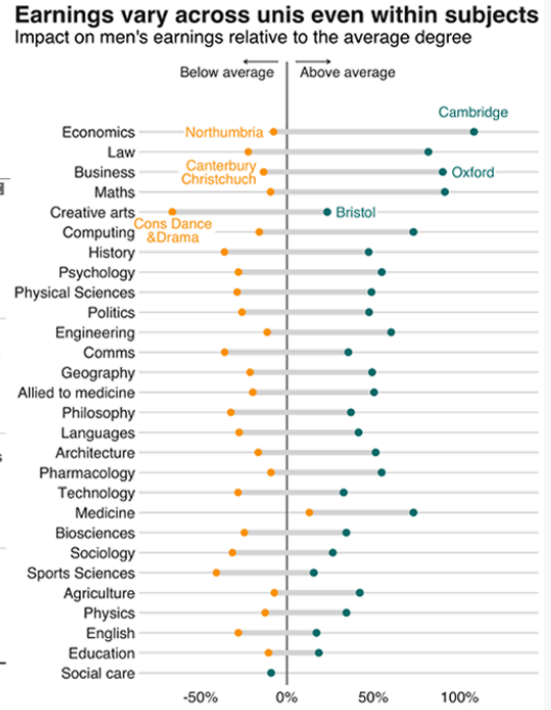
A ggplot showcase





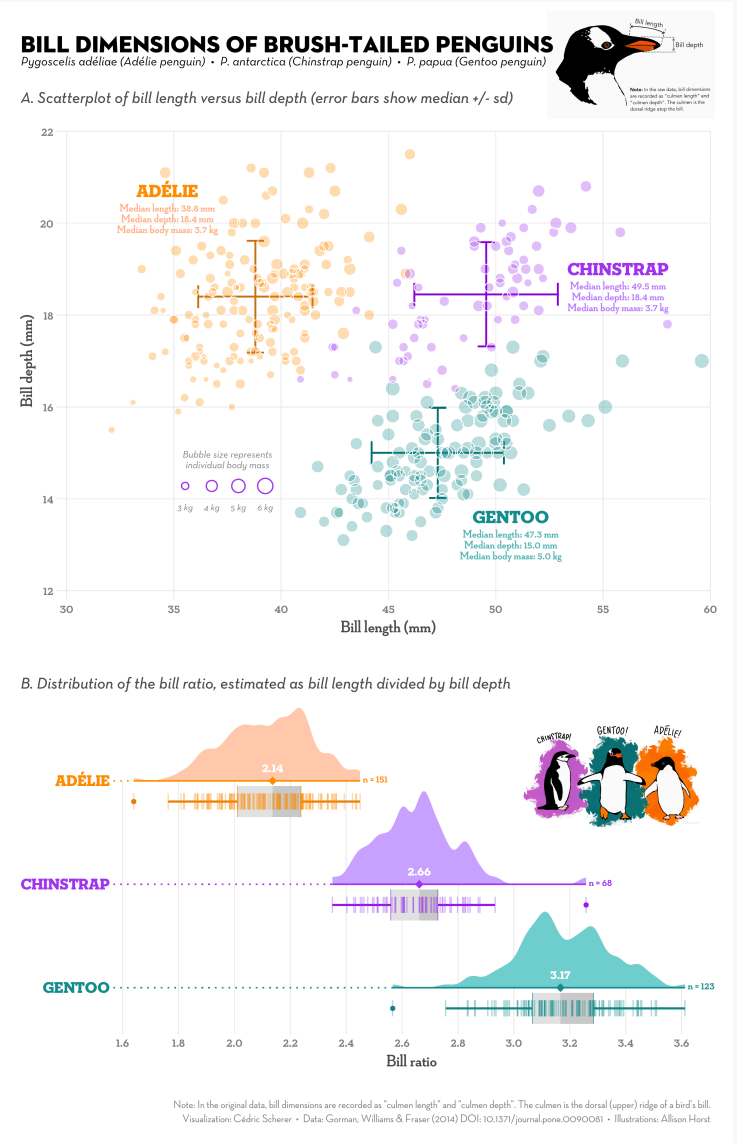






Visualizations produced by the BBC News data team

A ggplot showcase



Visualization by Cédric Scherer, code available on Github

The ggplot2 package

- **Consistent** grammar/structure
- **Flexible** structure allows you to produce any type of plots
- Highly **customizable appearance** (themes)
- Many **extension packages** that provide additional plot types, themes, colors, animation, …
  - See here for a list of ggplot extensions
- Active community that provides help and inspiration
- Perfect package for **exploratory data analysis** and **beautiful plots**

Part of the tidyverse

```
library(tidyverse)
# Or load it individually:
# library(ggplot2)
```

The data

The `gapminder` dataset contains data on life expectancy, GDP per capita, and population for 142 countries from 1952 to 2007.

```
# install.packages("gapminder")
library(gapminder)
gapminder
#> # A tibble: 1,704 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,694 more rows
```

```
gapminder_2007 <- filter(gapminder, year == 2007)
```

How is life expectancy related to GDP per capita?

Building a ggplot

Step by step

Step 1: `ggplot(data)`

The `ggplot()` function initializes a ggplot object. Every ggplot needs this function.

```
ggplot(gapminder_2007)
```

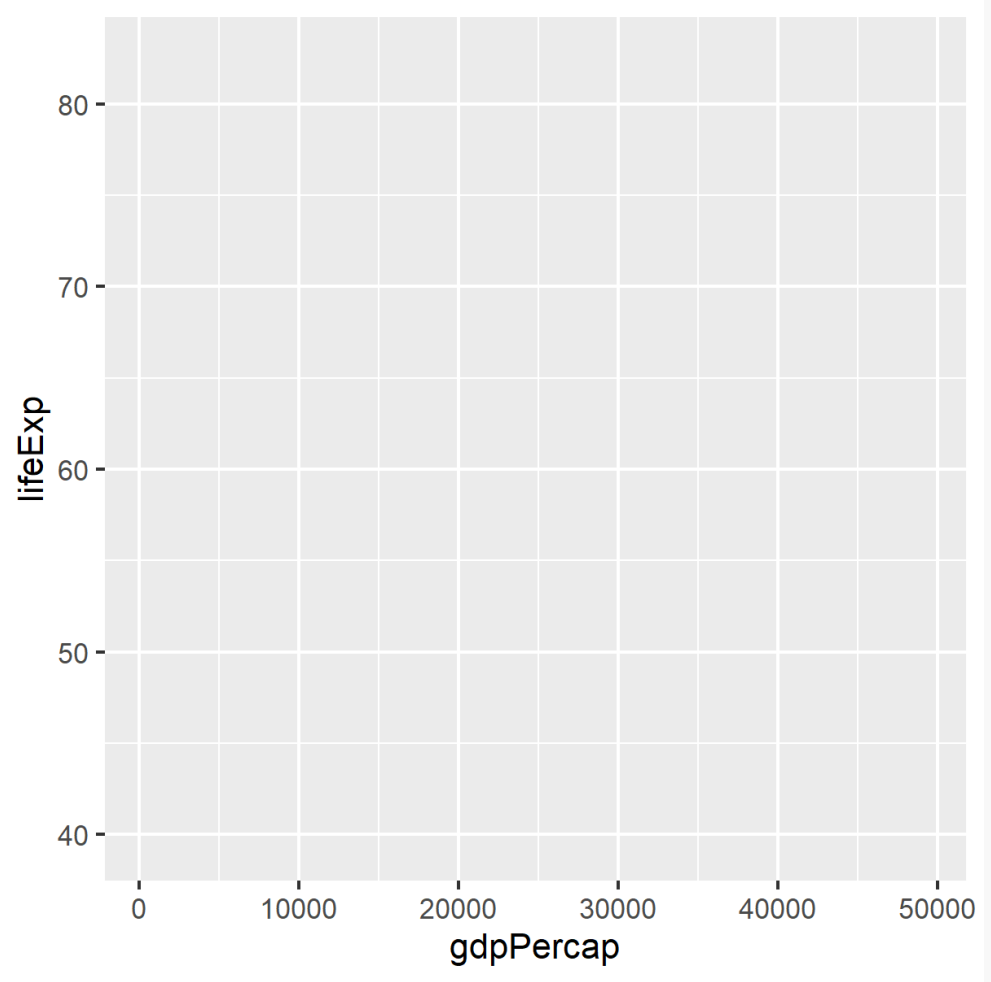
- Empty plot because we haven't told ggplot what to put on the axes yet

Step 2: add `aes(x, y)`

The **aes**thetics define how data variables are mapped to plot properties.

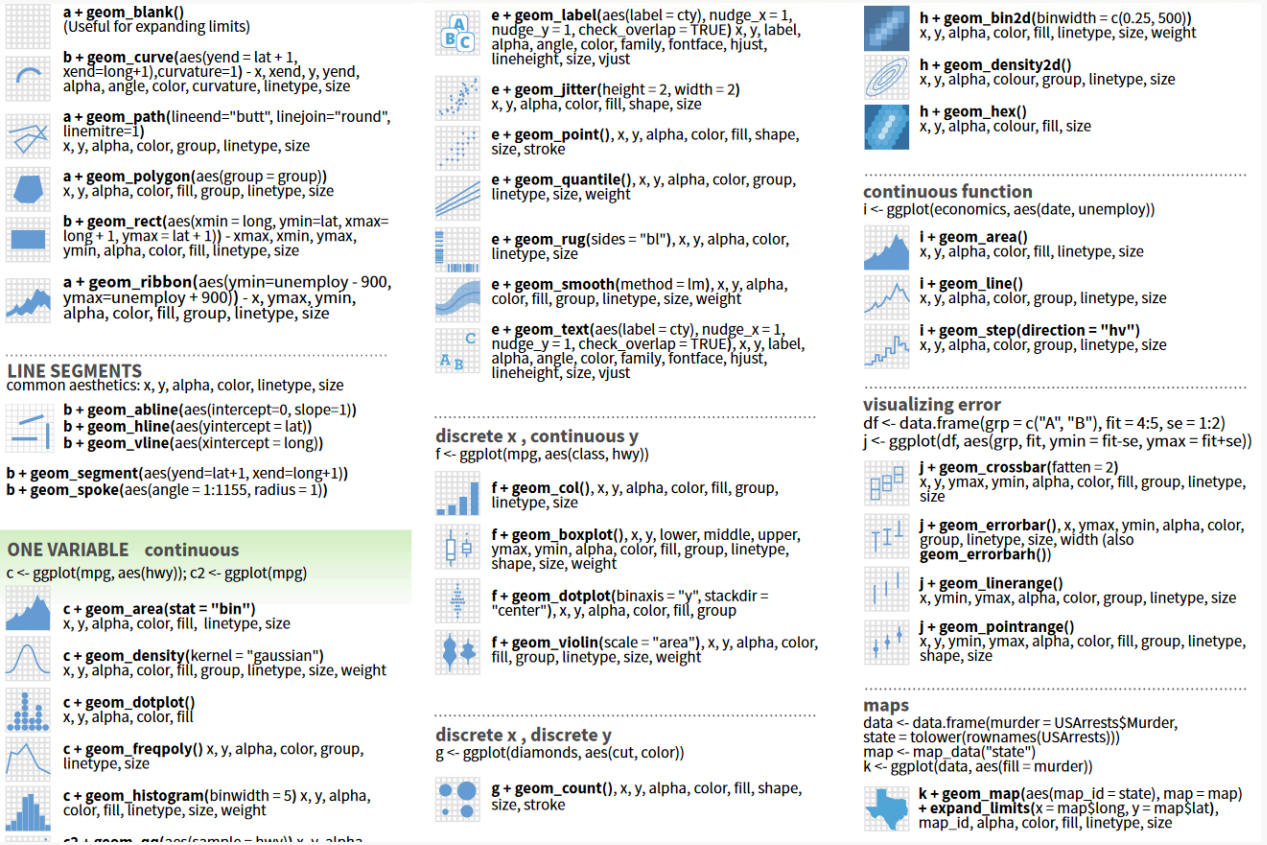
```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp
  )
)
```

- Axes are set up, but no data points yet
- Scales adapt automatically to the range of the data



Step 3: add a geom

geoms define how data points are represented. There are many different geoms to chose from



a + geom_blank()
(Useful for expanding limits)

b + geom_curve(aes(yend = lat + 1, xend=long+1),curvature=1) - x, xend, y, yend, alpha, angle, color, curvature, linetype, size

a + geom_path(lineend="butt", linejoin="round", linemitre=1)
x, y, alpha, color, group, linetype, size

a + geom_polygon(aes(group = group))
x, y, alpha, color, fill, group, linetype, size

b + geom_rect(aes(xmin = long, ymin=lat, xmax= long + 1, ymax = lat + 1)) - xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size

a + geom_ribbon(aes(ymin=unemploy - 900, ymax=unemploy + 900)) - x, ymax, ymin, alpha, color, fill, group, linetype, size

LINE SEGMENTS
common aesthetics: x, y, alpha, color, linetype, size

b + geom_abline(aes(intercept=0, slope=1))
b + geom_hline(aes(yintercept = lat))
b + geom_vline(aes(xintercept = long))

b + geom_segment(aes(yend=lat+1, xend=long+1))
b + geom_spoke(aes(angle = 1:1155, radius = 1))

ONE VARIABLE continuous
c <- ggplot(mpg, aes(hwy)); c2 <- ggplot(mpg)

c + geom_area(stat = "bin")
x, y, alpha, color, fill, linetype, size

c + geom_density(kernel = "gaussian")
x, y, alpha, color, fill, group, linetype, size, weight

c + geom_dotplot()
x, y, alpha, color, fill

c + geom_freqpoly() x, y, alpha, color, group, linetype, size

c + geom_histogram(binwidth = 5) x, y, alpha, color, fill, linetype, size, weight

e + geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE) x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

e + geom_jitter(height = 2, width = 2)
x, y, alpha, color, fill, shape, size

e + geom_point(), x, y, alpha, color, fill, shape, size, stroke

e + geom_quantile(), x, y, alpha, color, group, linetype, size, weight

e + geom_rug(sides = "bl"), x, y, alpha, color, linetype, size

e + geom_smooth(method = lm), x, y, alpha, color, fill, group, linetype, size, weight

e + geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

discrete x , continuous y
f <- ggplot(mpg, aes(class, hwy))

f + geom_col(), x, y, alpha, color, fill, group, linetype, size

f + geom_boxplot(), x, y, lower, middle, upper, ymax, ymin, alpha, color, fill, group, linetype, shape, size, weight

f + geom_dotplot(binaxis = "y", stackdir = "center"), x, y, alpha, color, fill, group

f + geom_violin(scale = "area"), x, y, alpha, color, fill, group, linetype, size, weight

discrete x , discrete y
g <- ggplot(diamonds, aes(cut, color))

g + geom_count(), x, y, alpha, color, fill, shape, size, stroke

h + geom_bin2d(binwidth = c(0.25, 500))
x, y, alpha, color, fill, linetype, size, weight

h + geom_density2d()
x, y, alpha, colour, group, linetype, size

h + geom_hex()
x, y, alpha, colour, fill, size

continuous function
i <- ggplot(economics, aes(date, unemploy))

i + geom_area()
x, y, alpha, color, fill, linetype, size

i + geom_line()
x, y, alpha, color, group, linetype, size

i + geom_step(direction = "hv")
x, y, alpha, color, group, linetype, size

visualizing error
df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)
j <- ggplot(df, aes(grp, fit, ymin = fit-se, ymax = fit+se))

j + geom_crossbar(fatten = 2)
x, y, ymax, ymin, alpha, color, fill, group, linetype, size

j + geom_errorbar(), x, ymax, ymin, alpha, color, group, linetype, size, width (also **geom_errorbarh()**)

j + geom_linerange()
x, ymin, ymax, alpha, color, group, linetype, size

j + geom_pointrange()
x, y, ymin, ymax, alpha, color, fill, group, linetype, shape, size

maps
data <- data.frame(murder = USArrests$Murder, state = tolower(rownames(USArrests)))
map <- map_data("state")
k <- ggplot(data, aes(fill = murder))

k + geom_map(aes(map_id = state), map = map) **+ expand_limits**(x = map$long, y = map$lat), map_id, alpha, color, fill, linetype, size

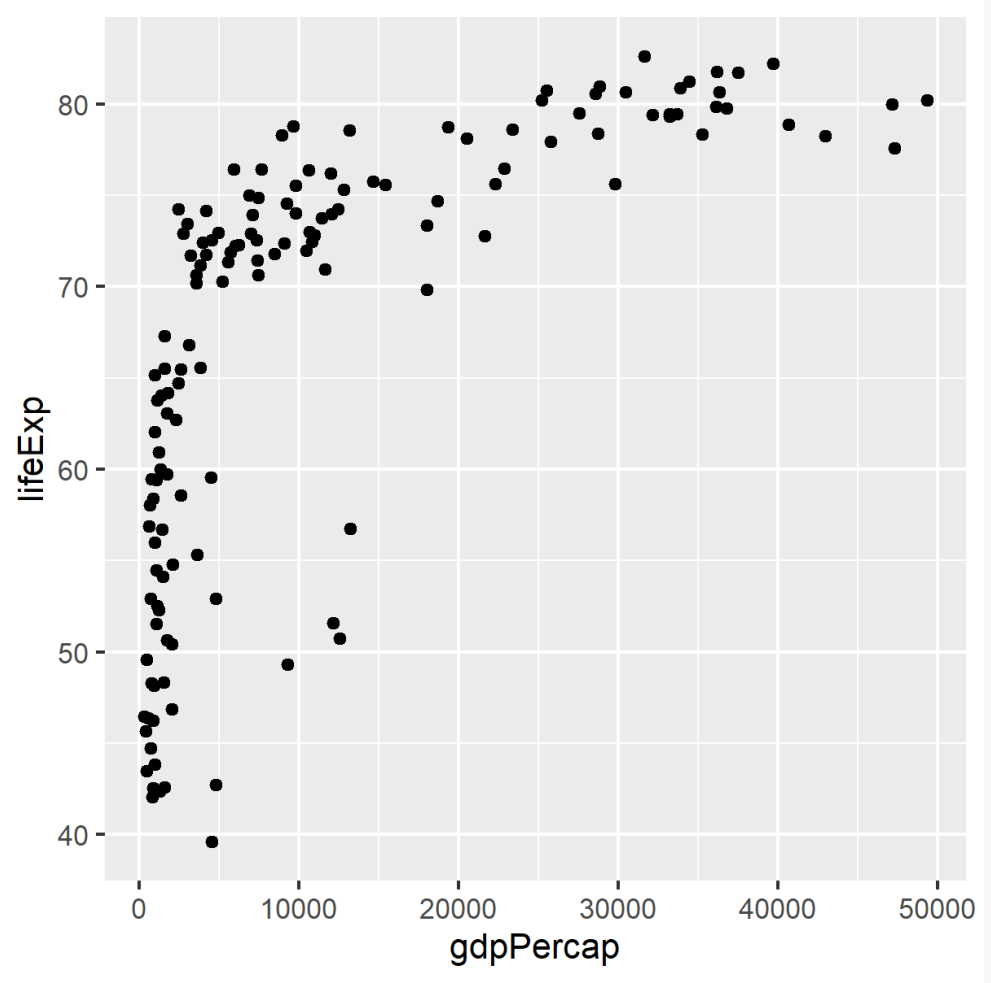
from ggplot cheatsheet

Step 3: add a geom

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp
  )
) +
  geom_point()
```

- New layers are added with `+`
- `data` and `aes` defined in `ggplot` call are inherited to all plot layers

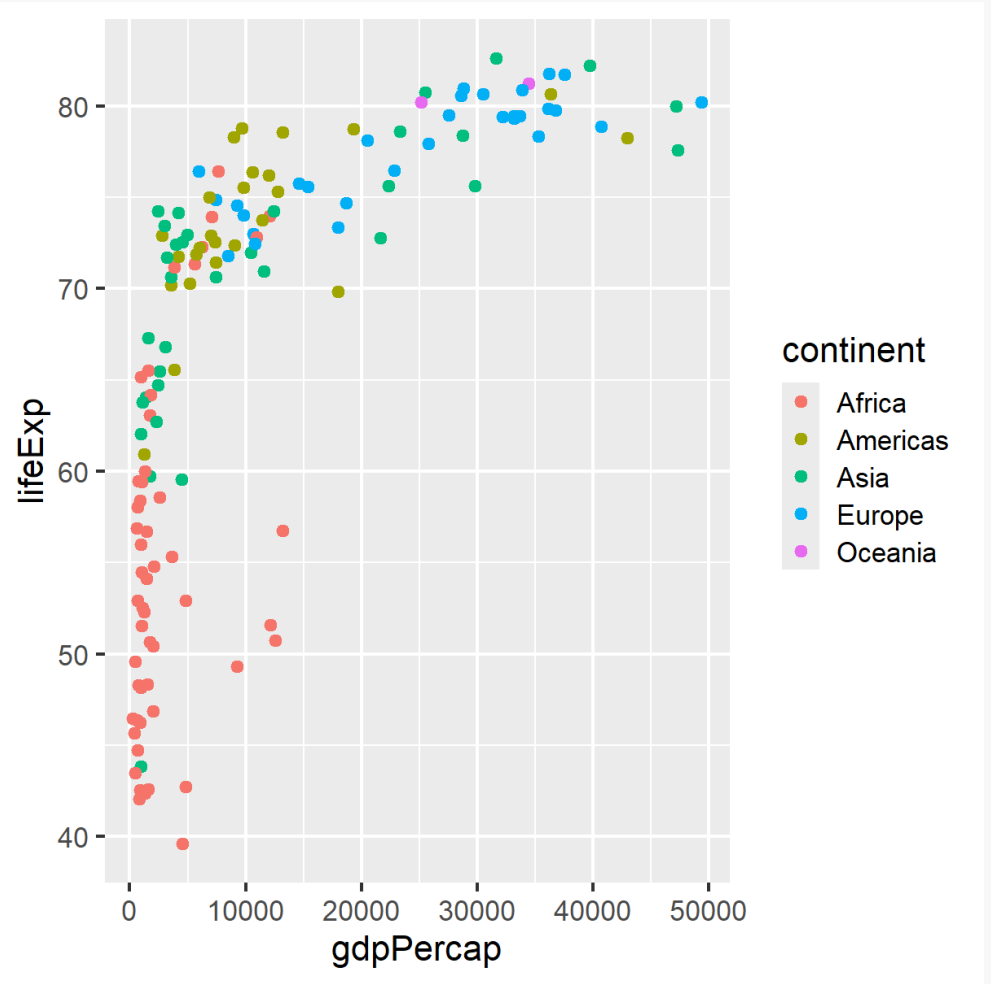
Does this depend on the continent?



Add color: `aes(color = ...)`

Map a variable to the **color** aesthetic to distinguish groups:

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point()
```

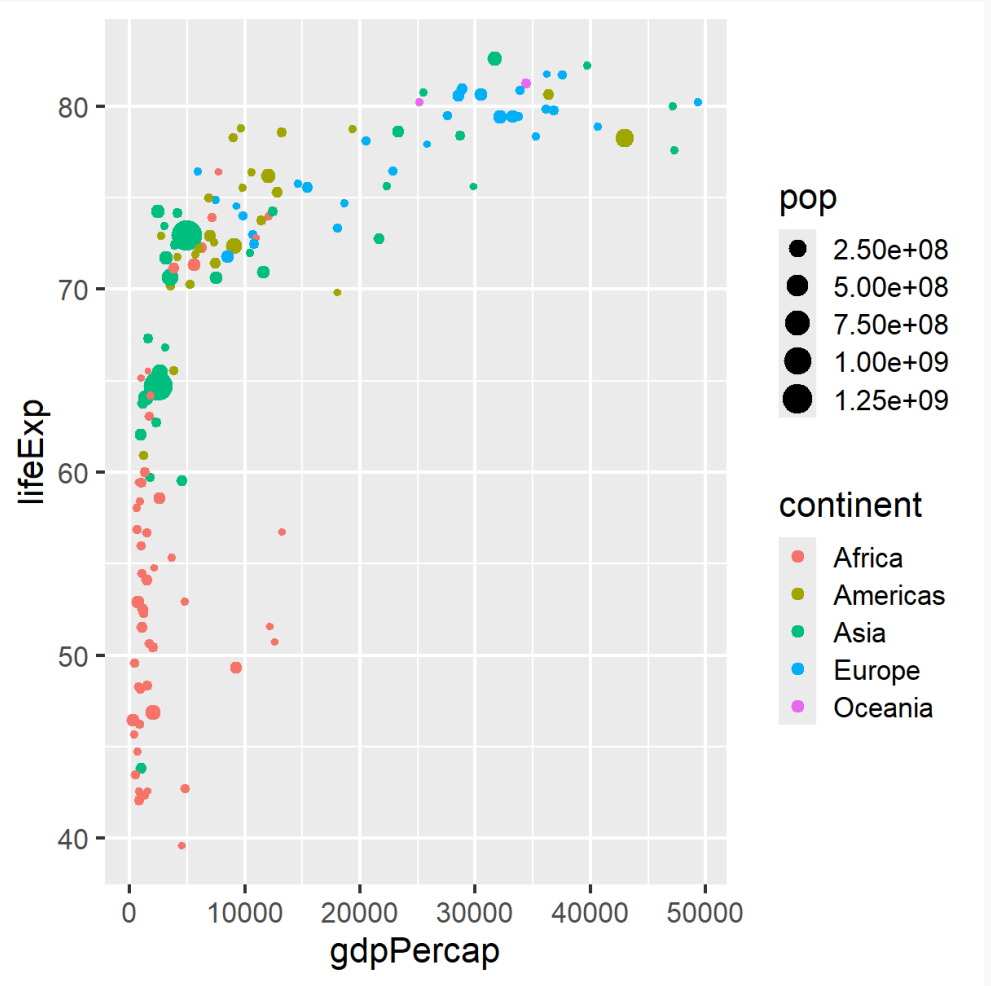


- `color = continent` maps the continent to point color
- A legend is added automatically

Other aesthetics: `size`

Besides `color`, you can also map variables to `size`:

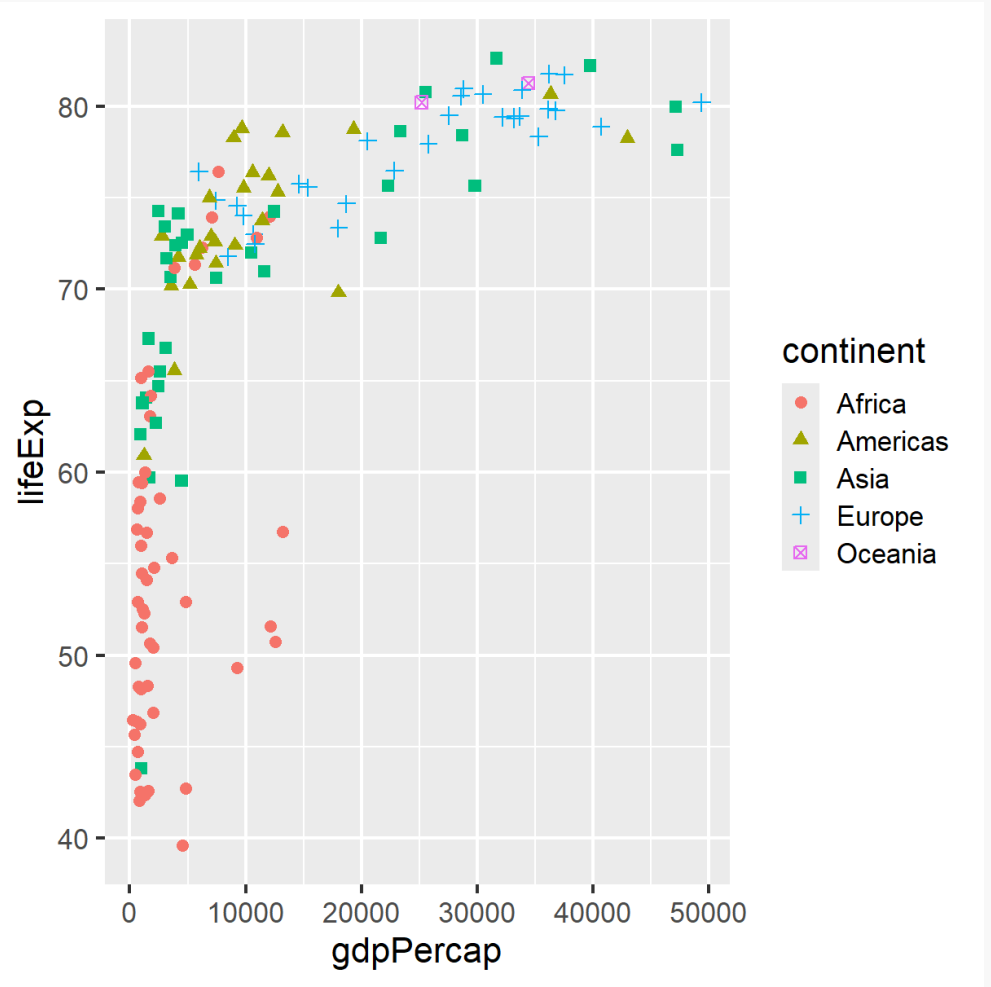
```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent,
    size = pop
  )
) +
  geom_point()
```



Other aesthetics: shape

Besides `color`, you can also map variables to `shape`:

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent,
    shape = continent
  )
) +
  geom_point()
```

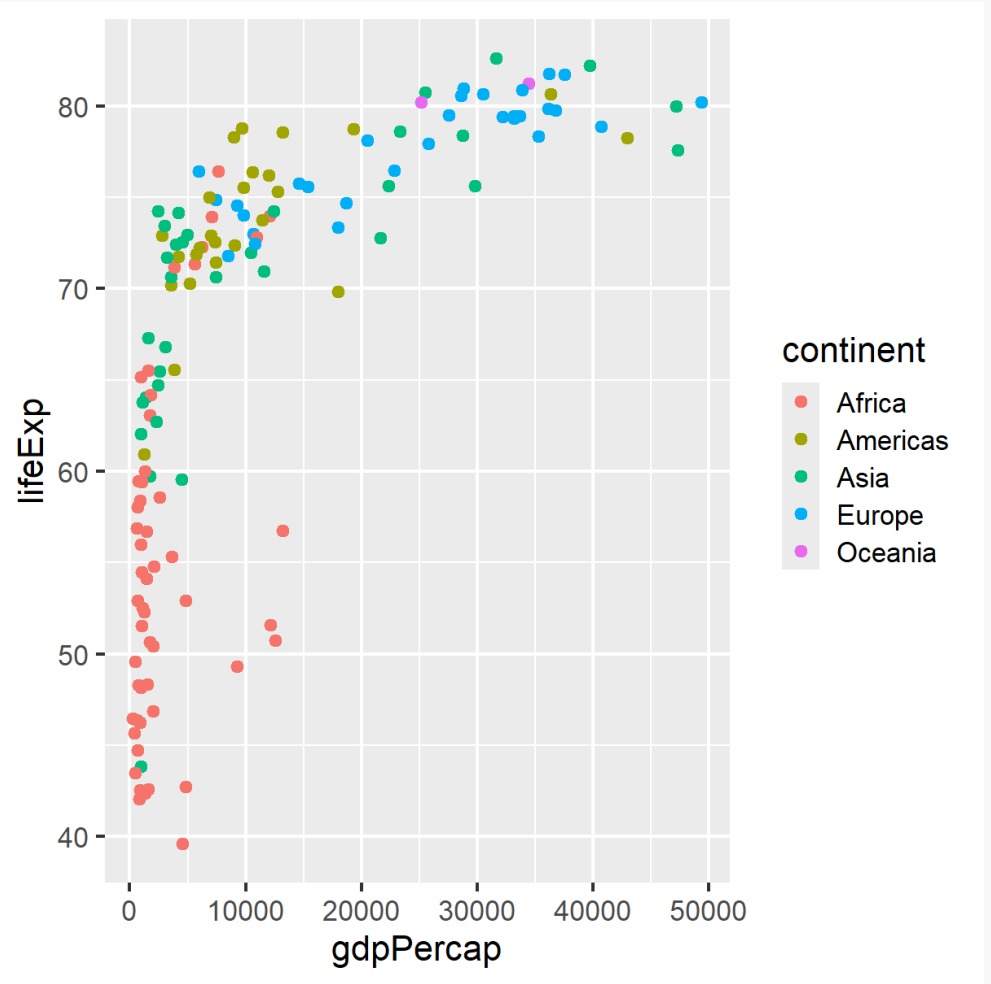


Changing the scales of the aesthetics

The scales onto which the aesthetic elements are mapped can be changed.

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point()
```

- GDP is condensed on the left (few large values)
- A log scale would spread the data out



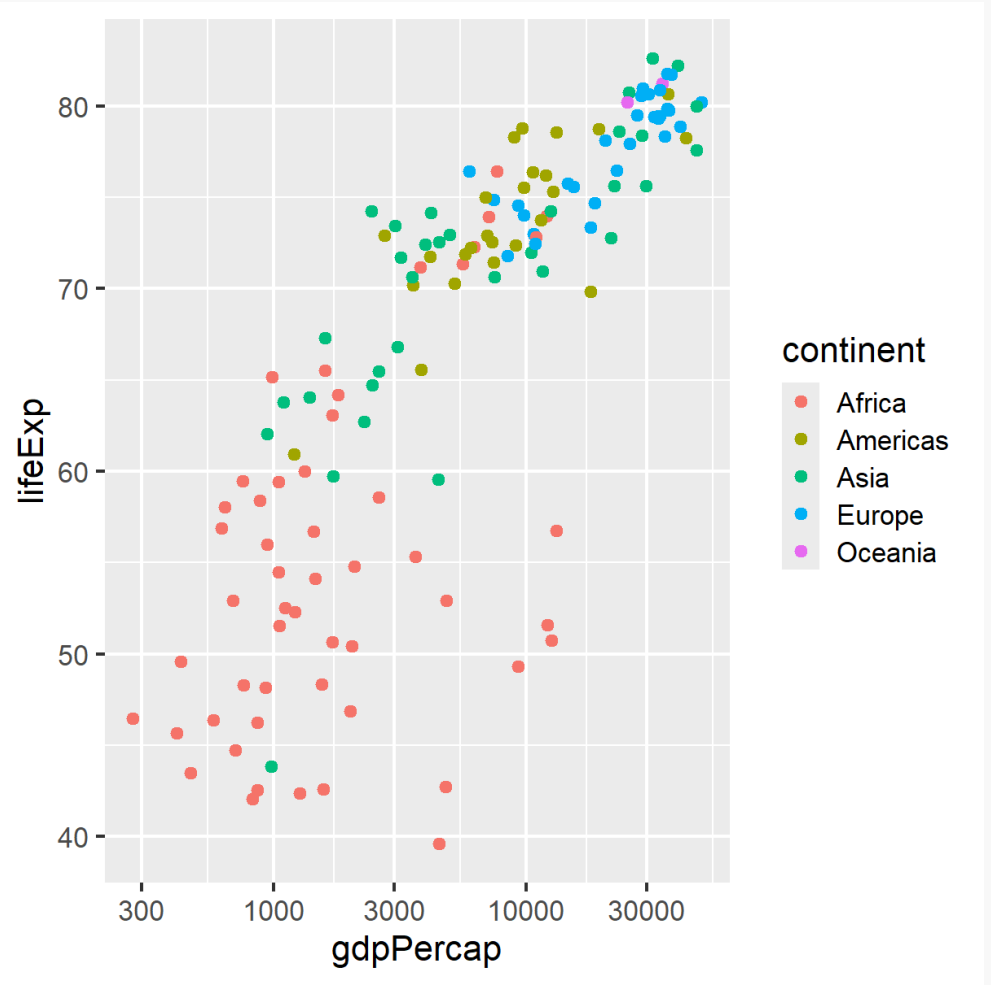
scale_x_log10

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point() +
  scale_x_log10()
```

- Scales can be changed for all elements of `aes`:

`scale_aes-name_scale-type`

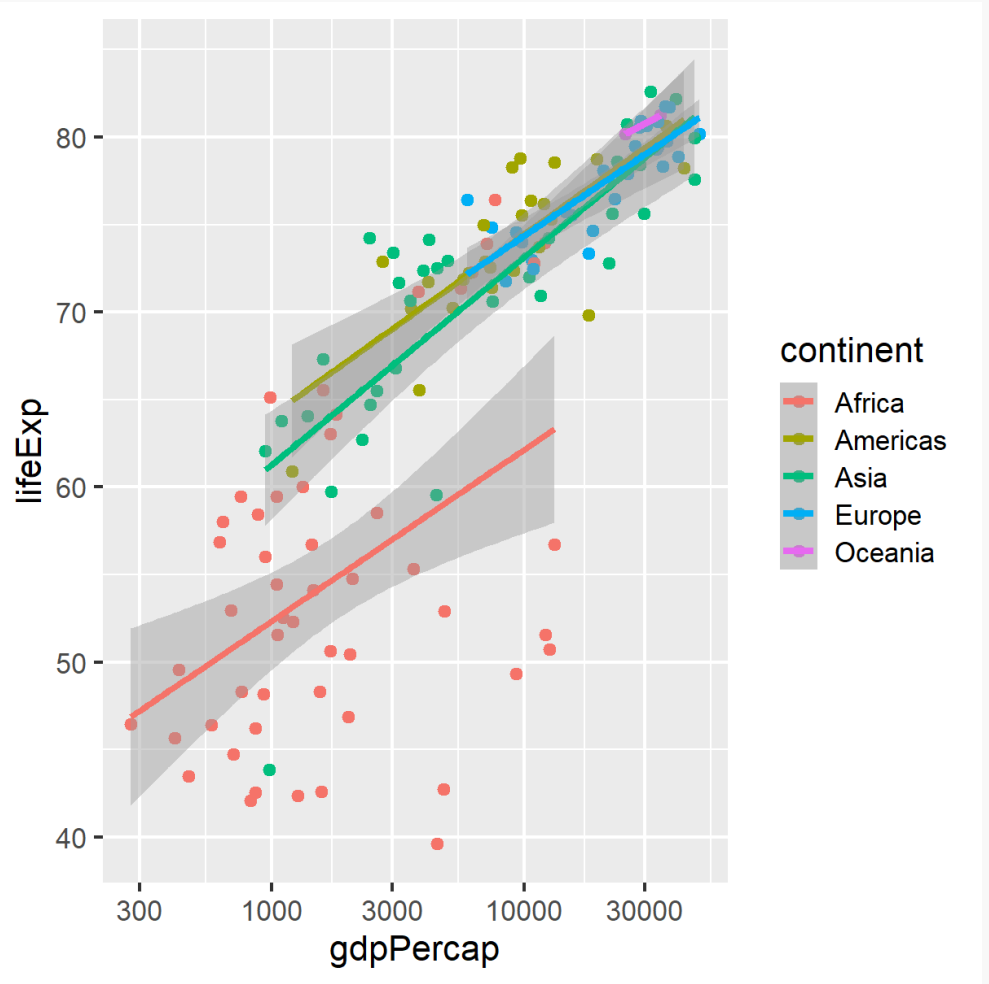
Here we scale the `x` aesthetic to `log10`.



geom_smooth

Add a smoothing line that helps see patterns in the data

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point() +
  geom_smooth(method = "lm") +
  scale_x_log10()
```

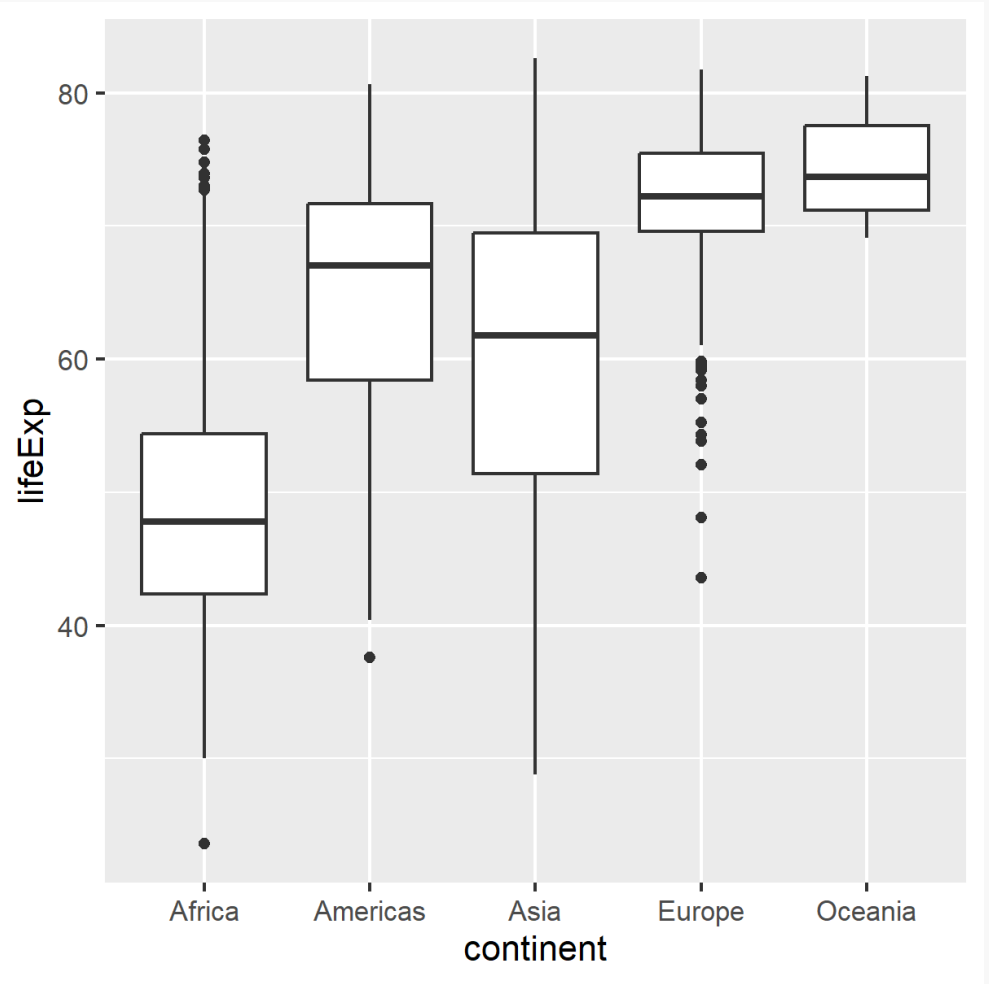


- With `method = "lm"`, a linear regression line is added
- All geoms done separately for each continent because color is defined globally

geom_boxplot

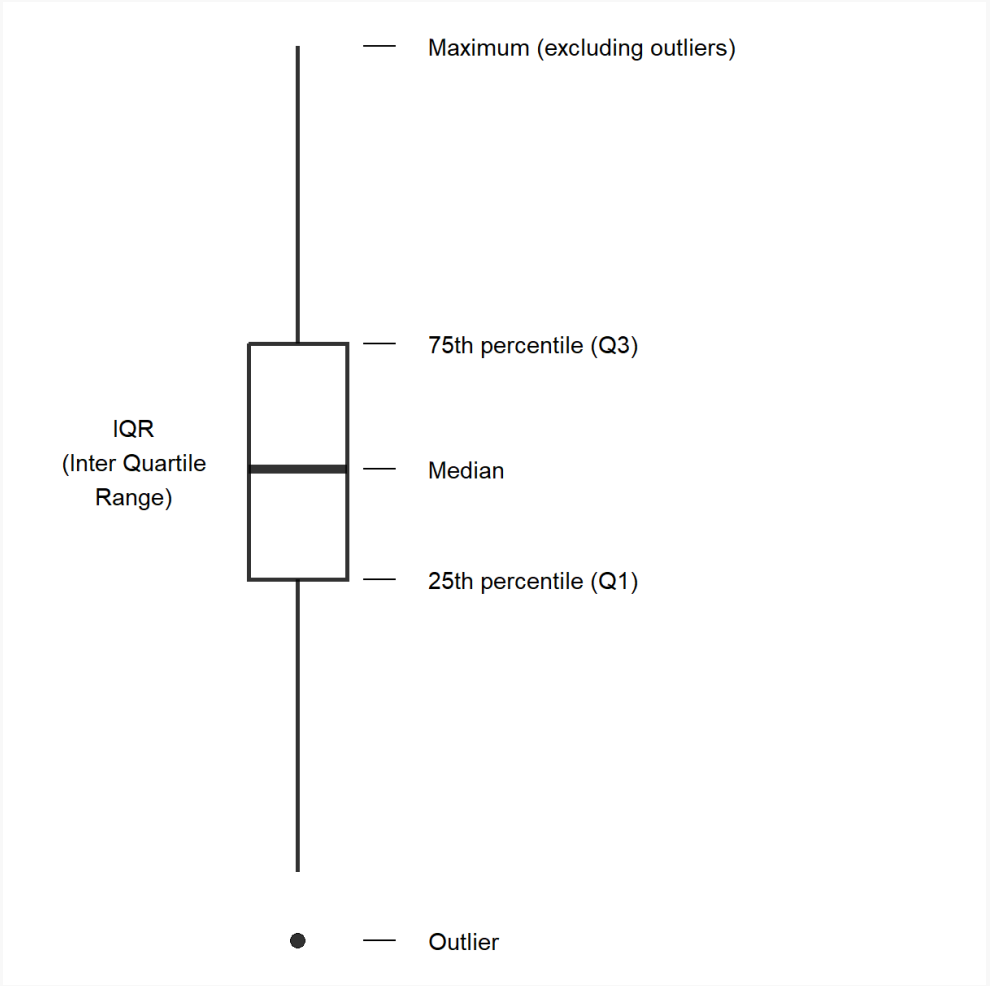
Compare a numeric variable across groups:

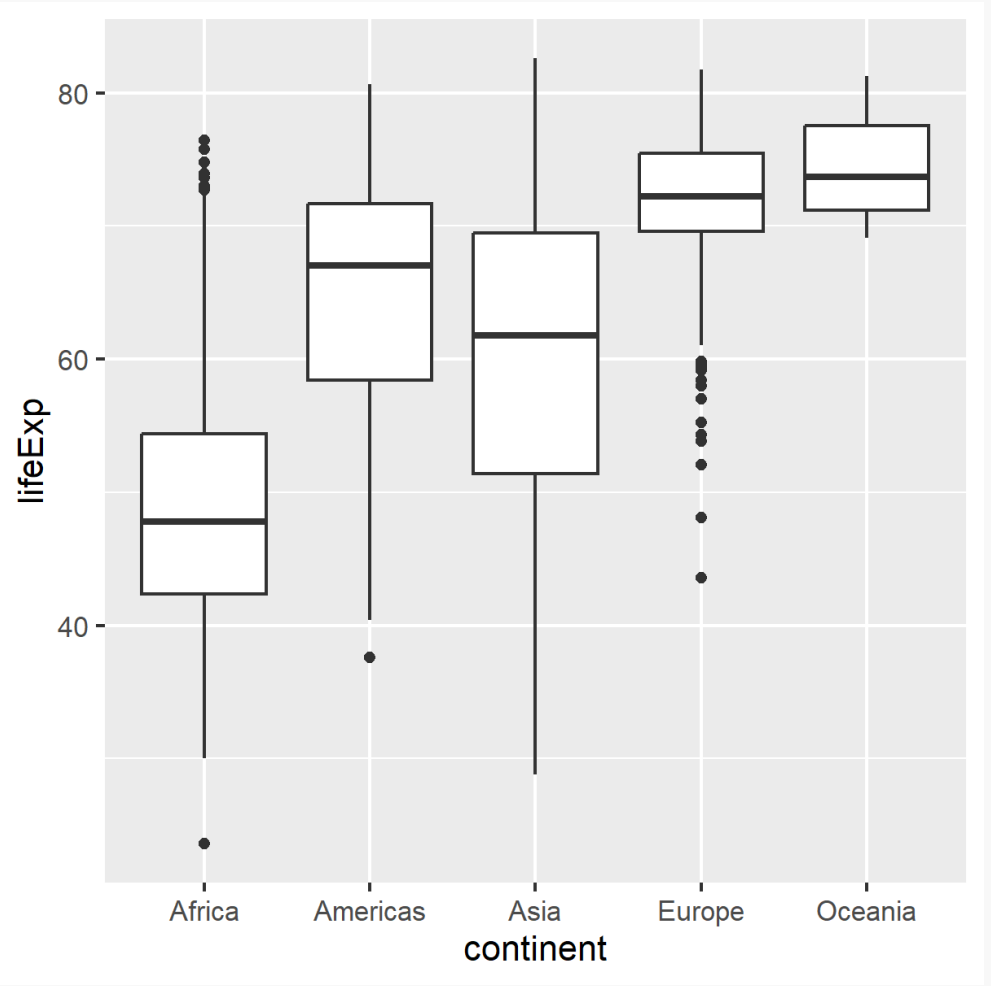
```
ggplot(
  gapminder,
  aes(
    x = continent,
    y = lifeExp
  )
) +
  geom_boxplot()
```



geom_boxplot

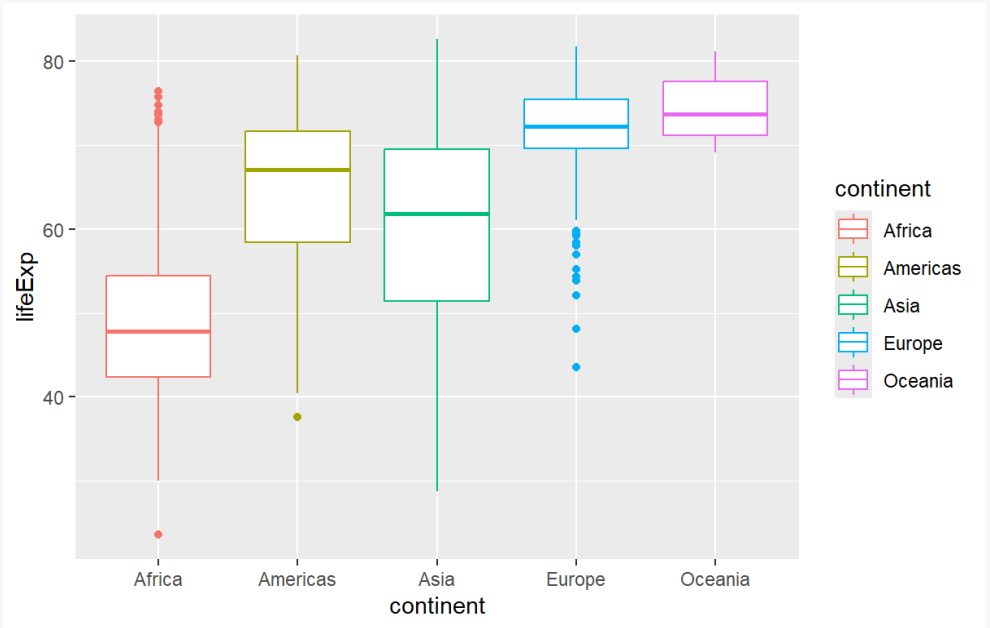
Compare a numeric variable across groups:



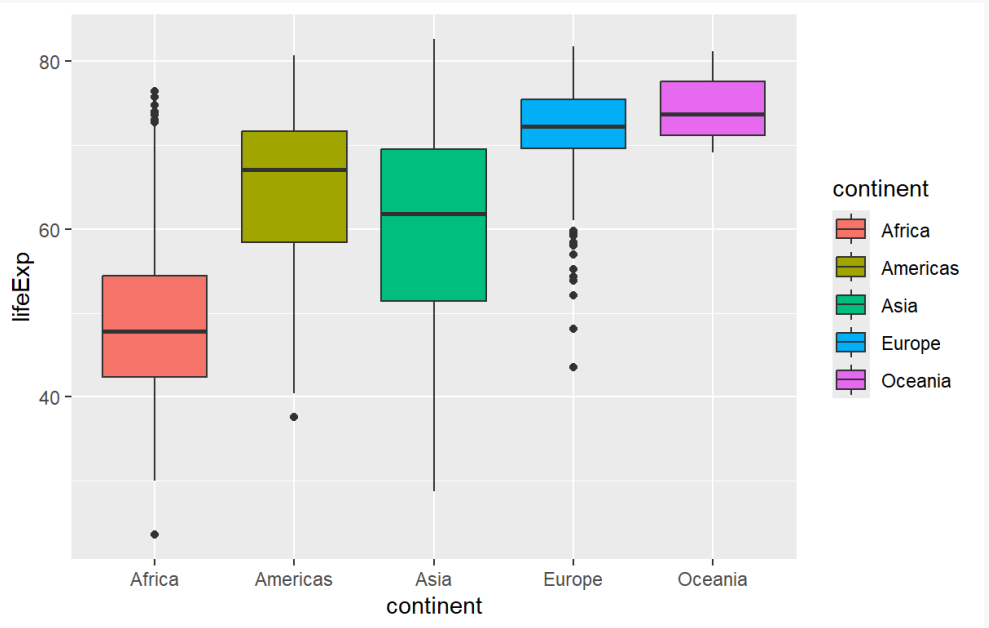


`geom_boxplot` with color or fill

```
ggplot(
  gapminder,
  aes(
    x = continent,
    y = lifeExp,
    color = continent
  )
) +
  geom_boxplot()
```




```
ggplot(
  gapminder,
  aes(
    x = continent,
    y = lifeExp,
    fill = continent
  )
) +
  geom_boxplot()
```

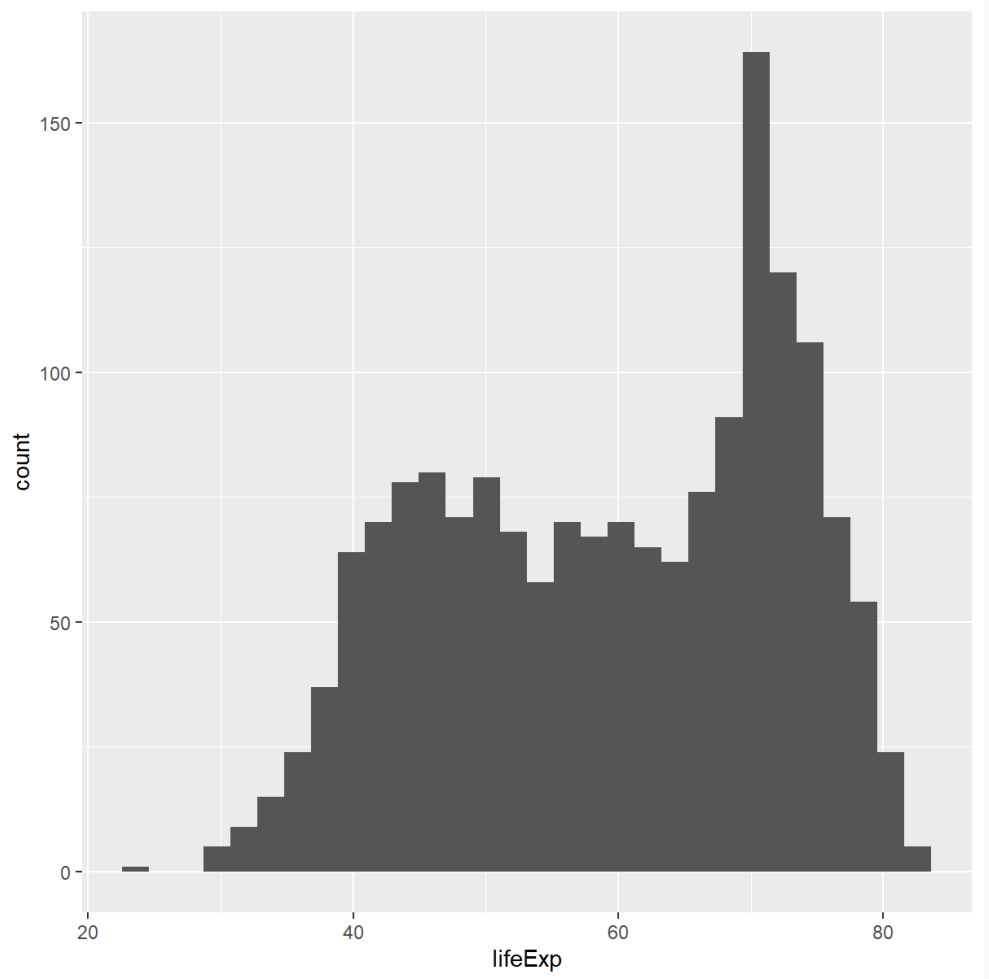


geom_histogram

See the distribution of a single numeric variable:

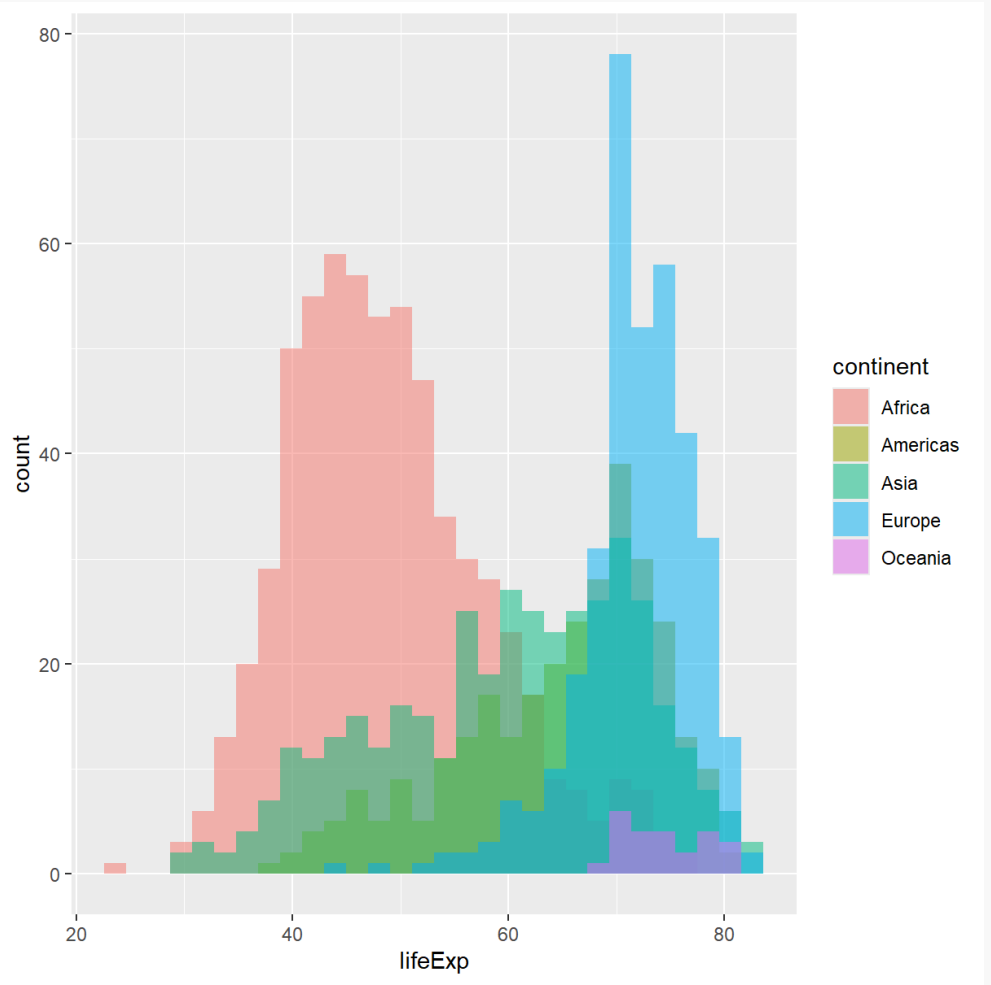
```
ggplot(
  gapminder,
  aes(x = lifeExp)
) +
  geom_histogram()
```

- ggplot counts the observations for you (y-axis)



`geom_histogram` with groups

```
ggplot(
  gapminder,
  aes(
    x = lifeExp,
    fill = continent
  )
) +
  geom_histogram(
    position = "identity",
    alpha = 0.5
  )
```

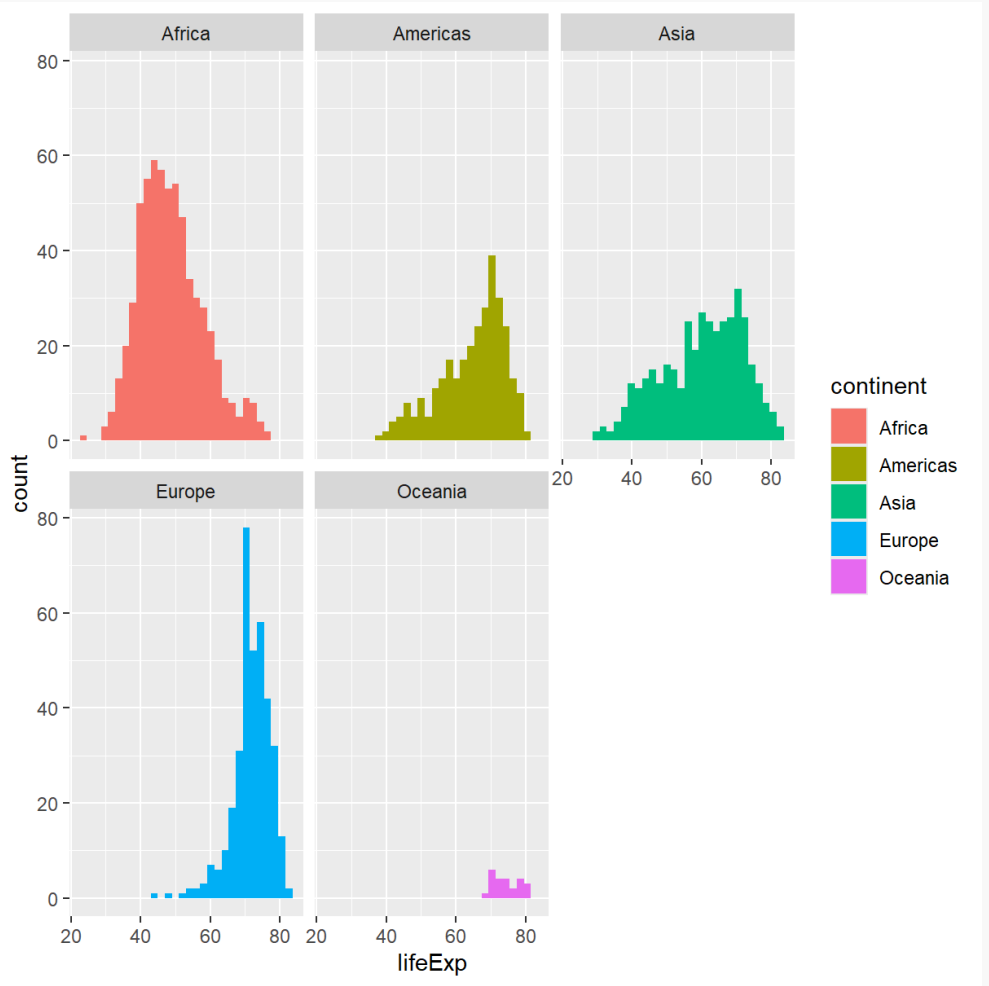


- By default histograms are stacked
- Use `position = "identity"` to overlap them
- `alpha` makes sure you see overlapping areas
- Here it's a bit too crowded to see differences between continents

Small multiples with `facet_wrap`

Split your plots along one variable with `facet_wrap`

```
ggplot(
  gapminder,
  aes(
    x = lifeExp,
    fill = continent
  )
) +
  geom_histogram() +
  facet_wrap(vars(continent))
```



- Especially useful when color/shape gets crowded
- Use `facet_grid()` to split along two variables

So many other geoms

Check out:

- ggplot2 cheatsheet
- ggplot2 cookbook
- ggplot2 tutorial

Summary: The ggplot skeleton

Every ggplot has the same basic structure:

```
ggplot(
  data,
  aes(x = ..., y = ..., color = ...)
) +
  geom_xxx()
```

- `ggplot()`: initialize the plot with data and aesthetic mappings
- `aes()`: map variables to visual properties (x-axis, y-axis, color, shape, size, ...)
- `geom_xxx()`: define how data points are drawn
- Layers are connected with `+`

Now you

Task 1 (35 min)

Exploratory data analysis with the penguin data set

Find the task description here

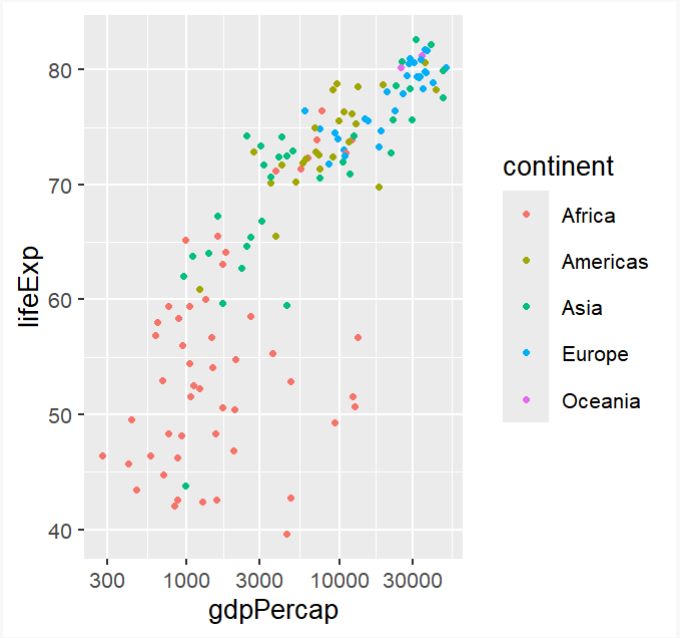
Beautifying plots

From default to publication-ready

Mapping vs. setting aesthetics

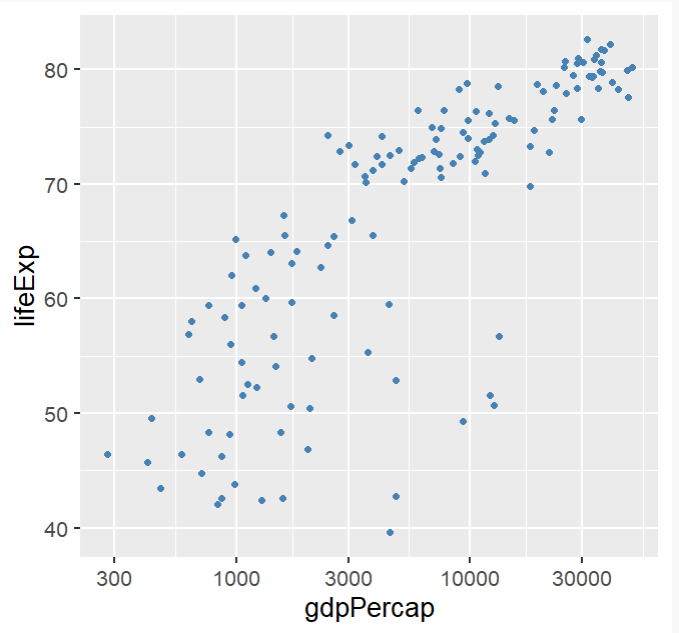
Inside `aes()`: **map** a variable to color

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point()
```



Inside `geom`: **set** color of all points

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp
  )
) +
  geom_point(
    color = "steelblue"
  )
```

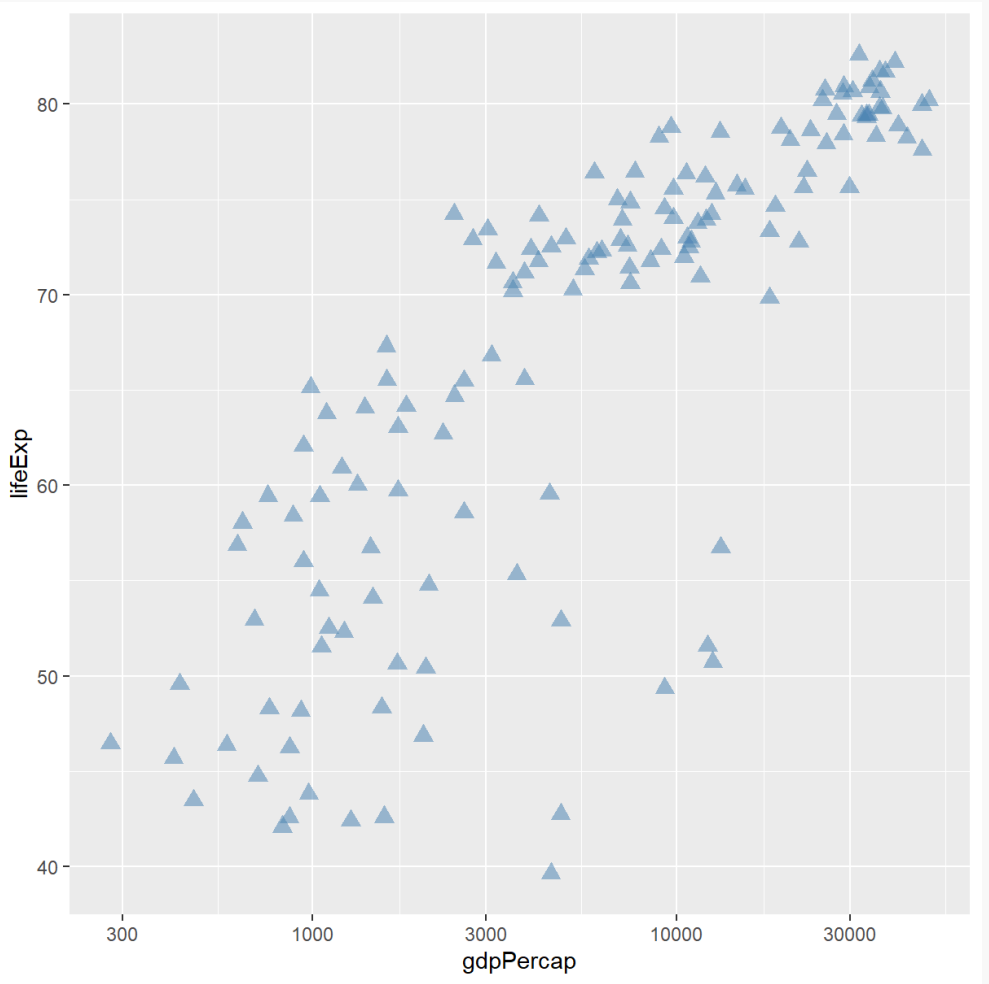


Mapping vs. setting aesthetics

You can change point shape, size, and transparency in the same way:

```
ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp
  )
) +
  geom_point(
    size = 3, # size
    shape = 17, # shape
    color = "steelblue", # color
    alpha = 0.5 # transparency (0-1)
  ) +
  scale_x_log10()
```



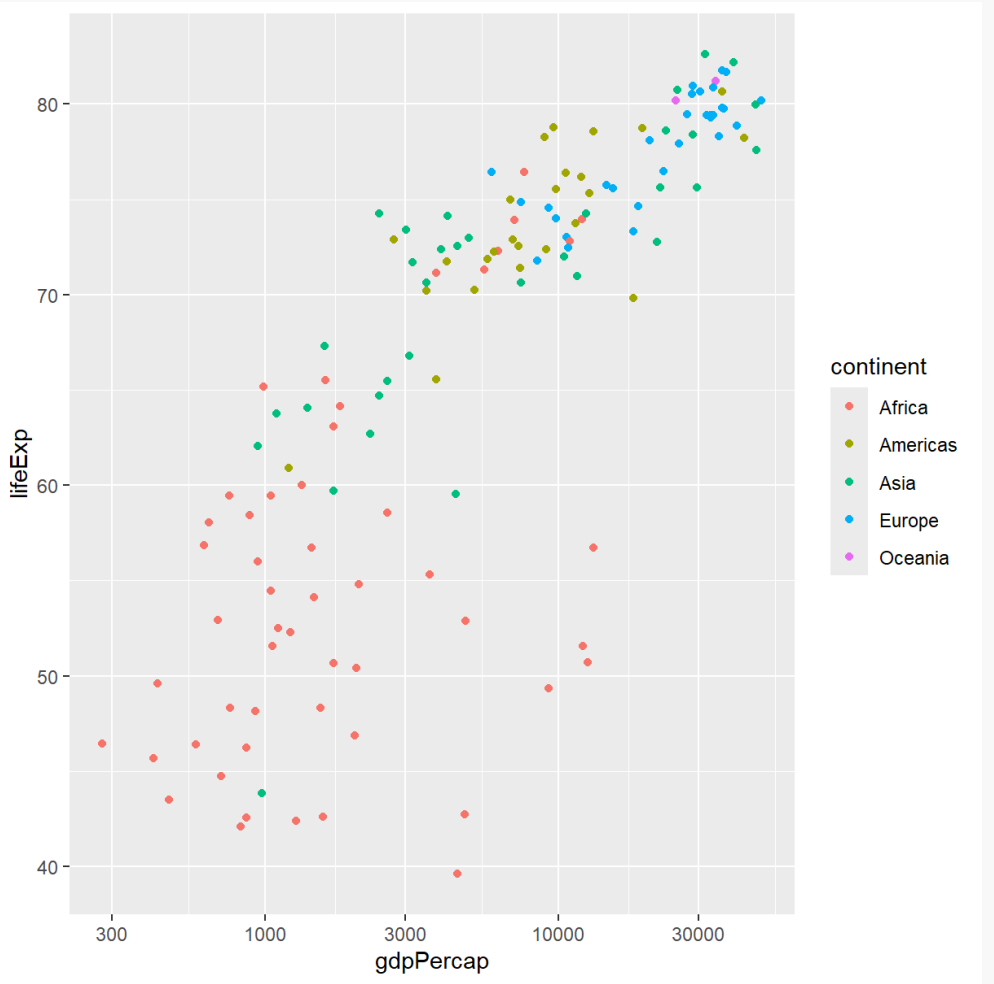


Shape and color codes

Starting point

Let's take this plot and make it look better step by step:

```
my_plot <- ggplot(
  gapminder_2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    color = continent
  )
) +
  geom_point() +
  scale_x_log10()
my_plot
```

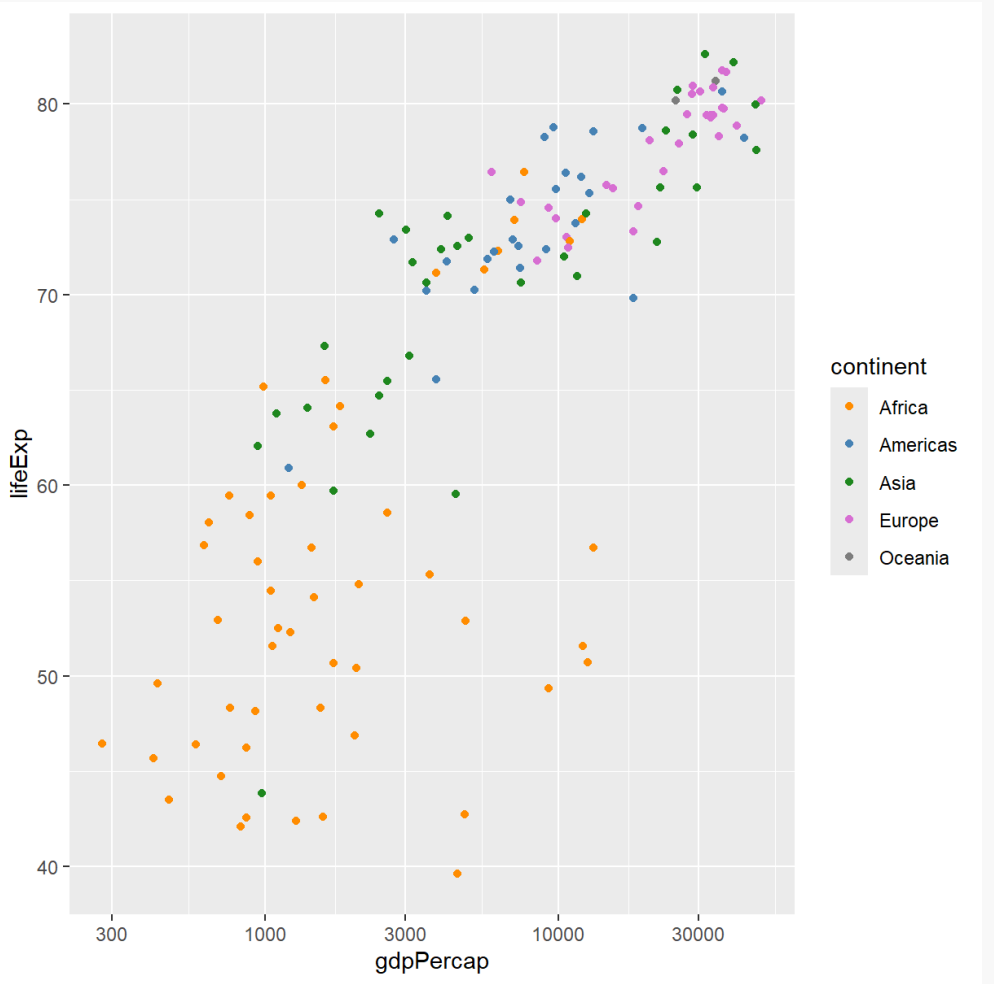


- Saving the plot in a variable `my_plot` so we can build on it
- Other plot layers can still be added to `my_plot`

`scale_color_manual()`: choose your own colors

Change the colors of the color aesthetic:

```
my_plot <- my_plot +
  scale_color_manual(
    values = c(
      "darkorange",
      "steelblue",
      "forestgreen",
      "orchid",
      "grey70"
    )
  )
my_plot
```



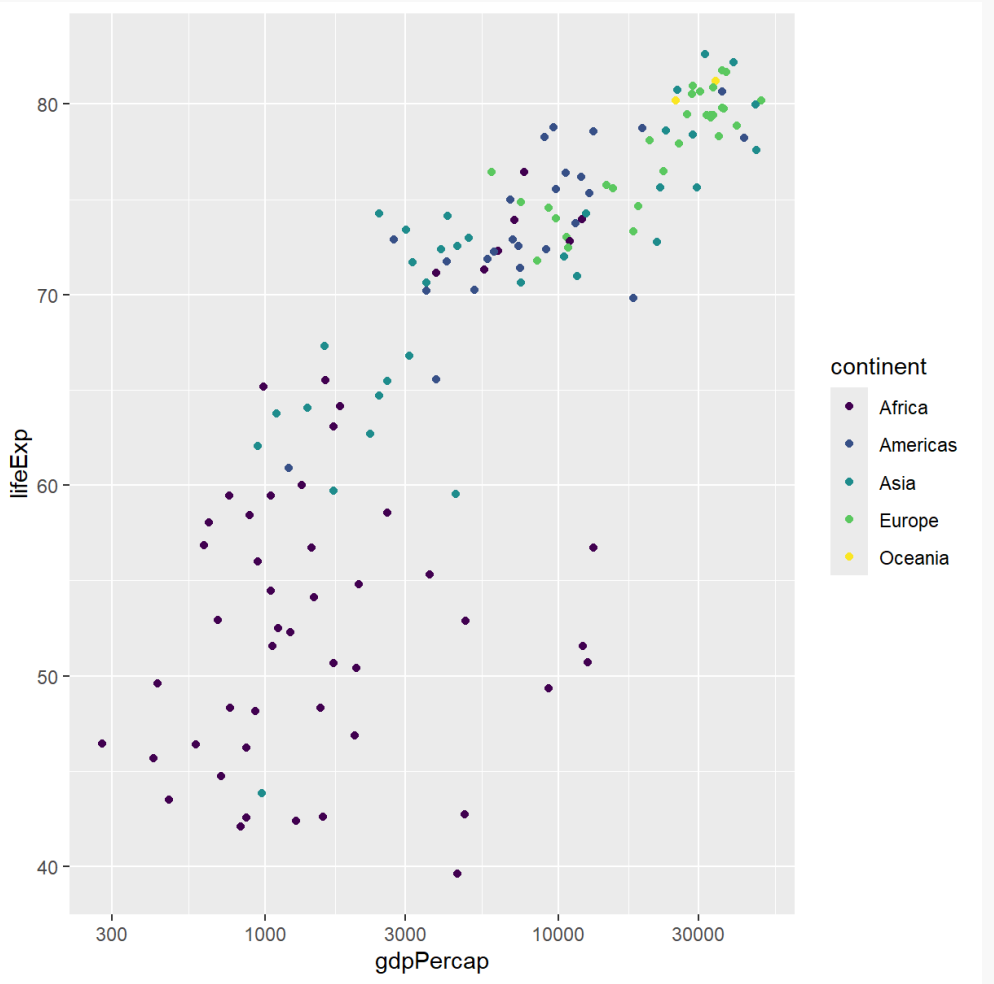
- One color per group: number of colors must match the number of groups
- Colors can be names or hex codes like `"#FF6B35"`

scale_color_viridis_d

Change the colors of the color aesthetic:

```
my_plot +
  scale_color_viridis_d()
```

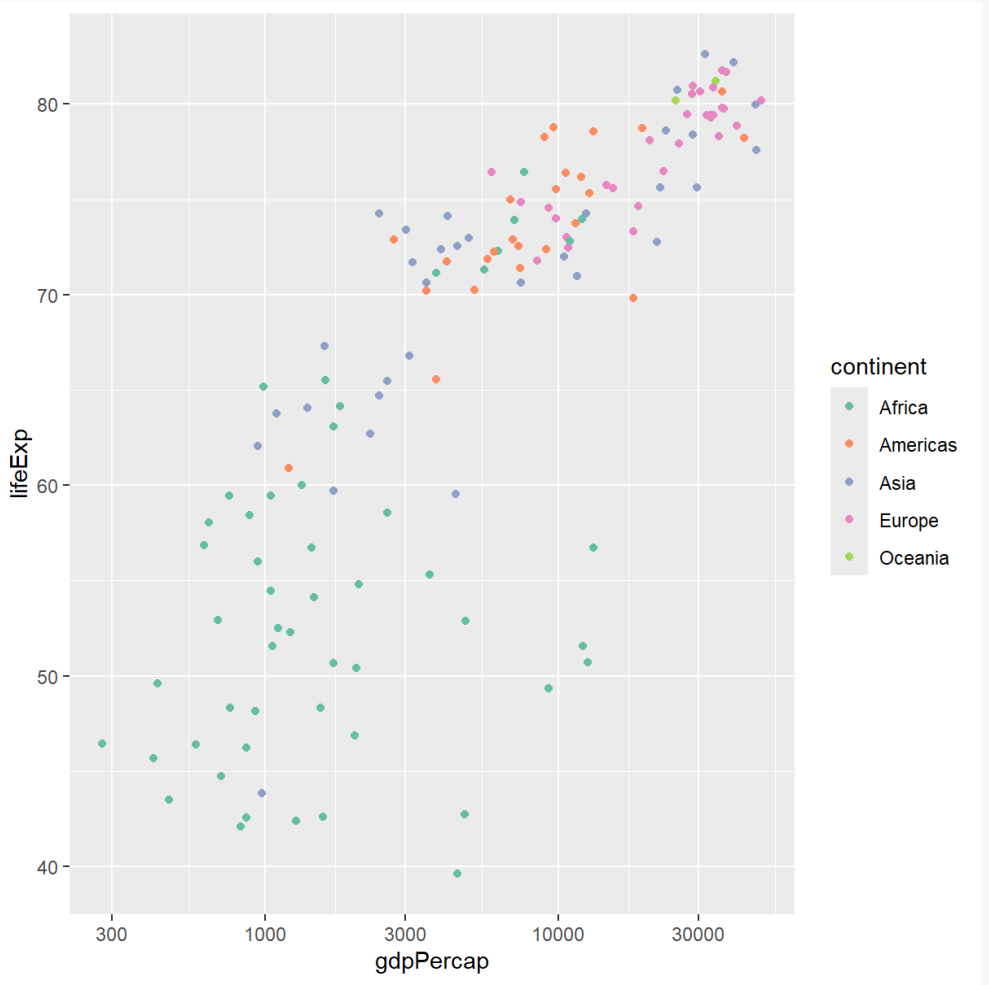
- The viridis color palette is designed for viewers with common forms of color blindness
- `viridis_d` for discrete colors, `viridis_c` for continuous colors
- Other viridis palettes with the `option` argument: `"magma"`, `"inferno"`, `"plasma"`, `"cividis"`



`scale_color_brewer()`: use a color palette

```
my_plot +
  scale_color_brewer(
    palette = "Set2"
  )
```

- Pre-defined palettes, many are colorblind-friendly
- Other options: `"Dark2"`, `"Paired"`, `"Pastel1"`, …
- Check `?scale_color_brewer` for all options



So many color options

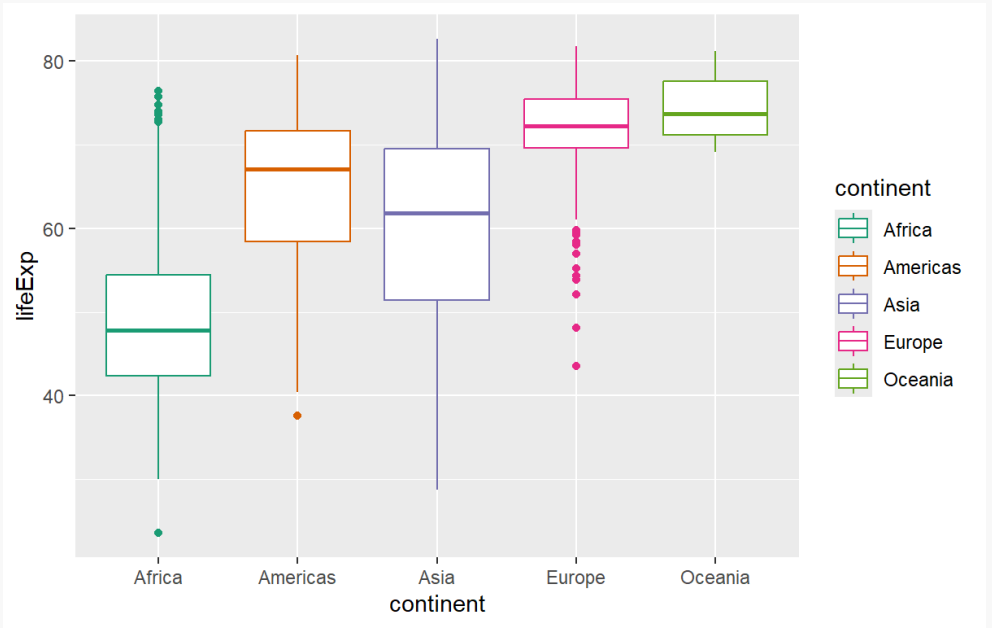
Many packages provide additional color palettes for ggplot:

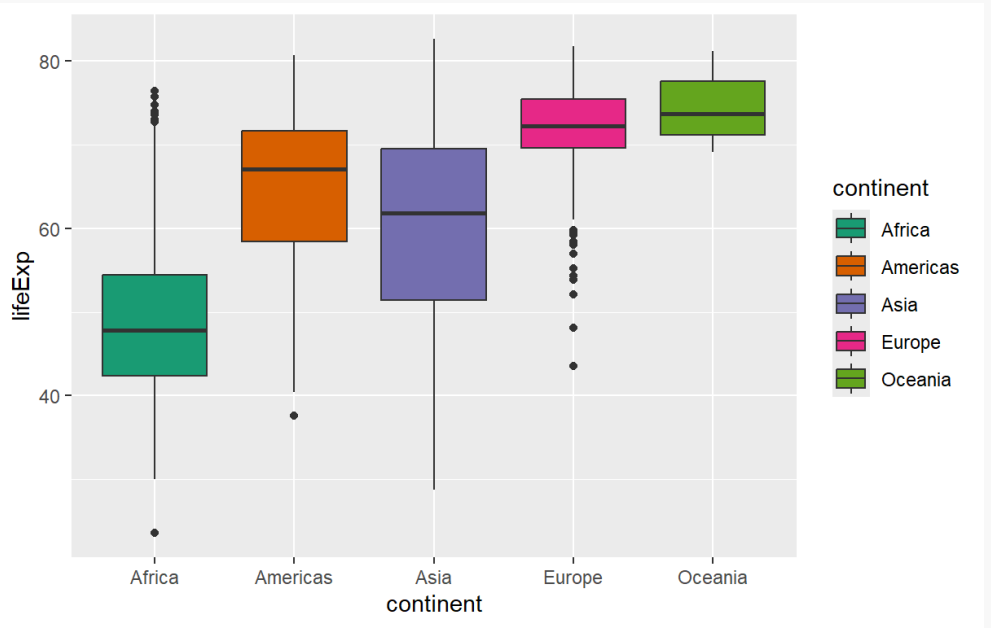
- ggsci: palettes inspired by scientific journals (e.g. Nature, Lancet) and sci-fi movies
- scico: scientific color palettes that are colorblind-friendly
- MetBrewer: palettes inspired by paintings from the Metropolitan Museum of Art
- NatParksPalettes: palettes inspired by photos of national parks from around the world
- ggthemes: palettes and themes from e.g. The Economist, Tableau, Wall Street Journal
- paletteer: one interface to thousands of palettes from many different packages

scale_fill_* vs. scale_color_*

```
ggplot(
  gapminder,
  aes(
    x = continent,
    y = lifeExp,
    color = continent
  )
) +
  geom_boxplot() +
  scale_color_brewer(
    palette = "Dark2"
  )
```

```
ggplot(
  gapminder,
  aes(
    x = continent,
    y = lifeExp,
    fill = continent
  )
) +
  geom_boxplot() +
  scale_fill_brewer(
    palette = "Dark2"
  )
```

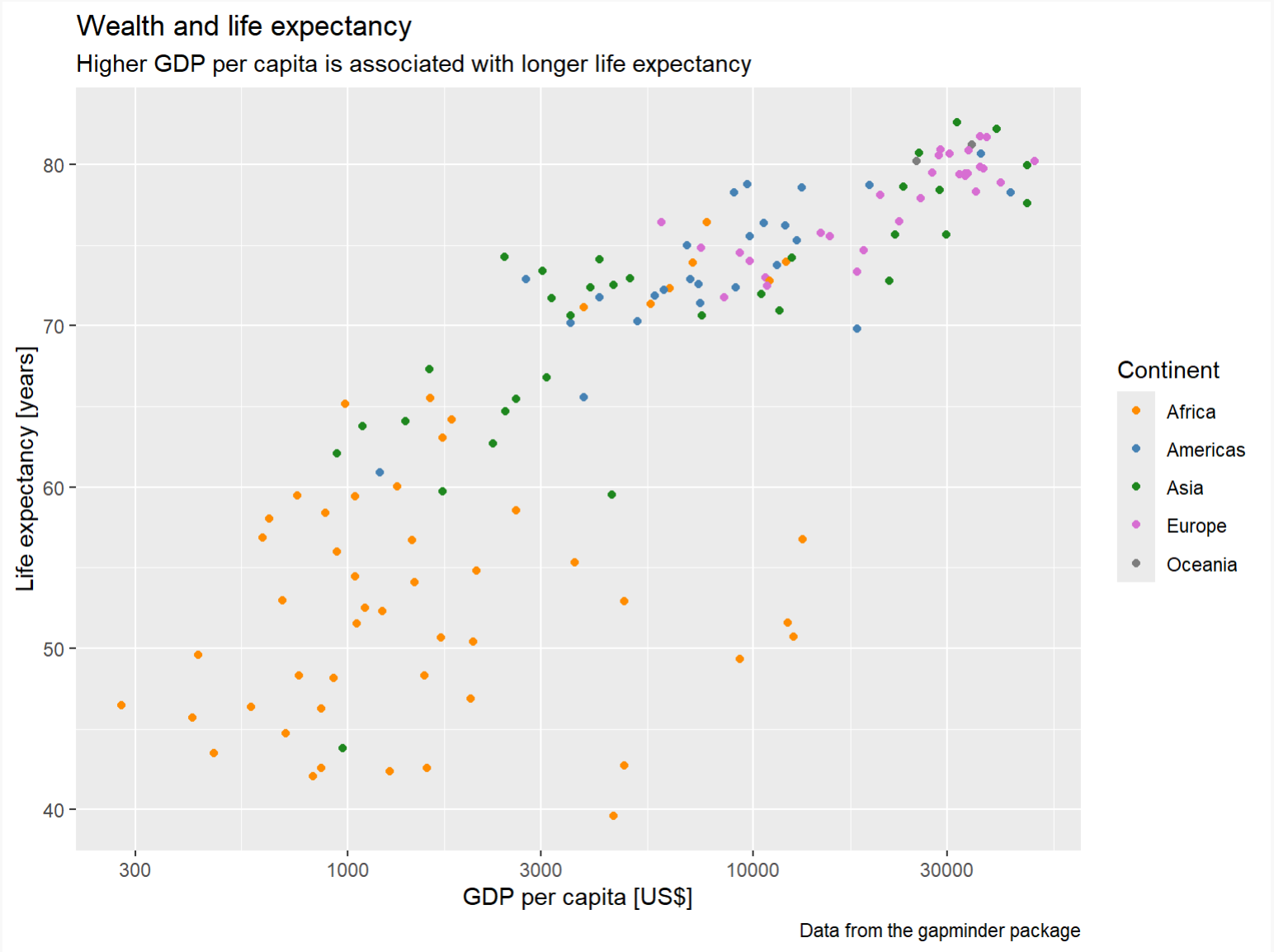




`labs`: Change axis and legend titles and add plot title

```
my_plot <- my_plot +
  labs(
    x = "GDP per capita [US$]",
    y = "Life expectancy [years]",
    color = "Continent",
    title = "Wealth and life expectancy",
    subtitle = "Higher GDP per capita is associated with longer life expectancy",
    caption = "Data from the gapminder package"
  )
my_plot
```

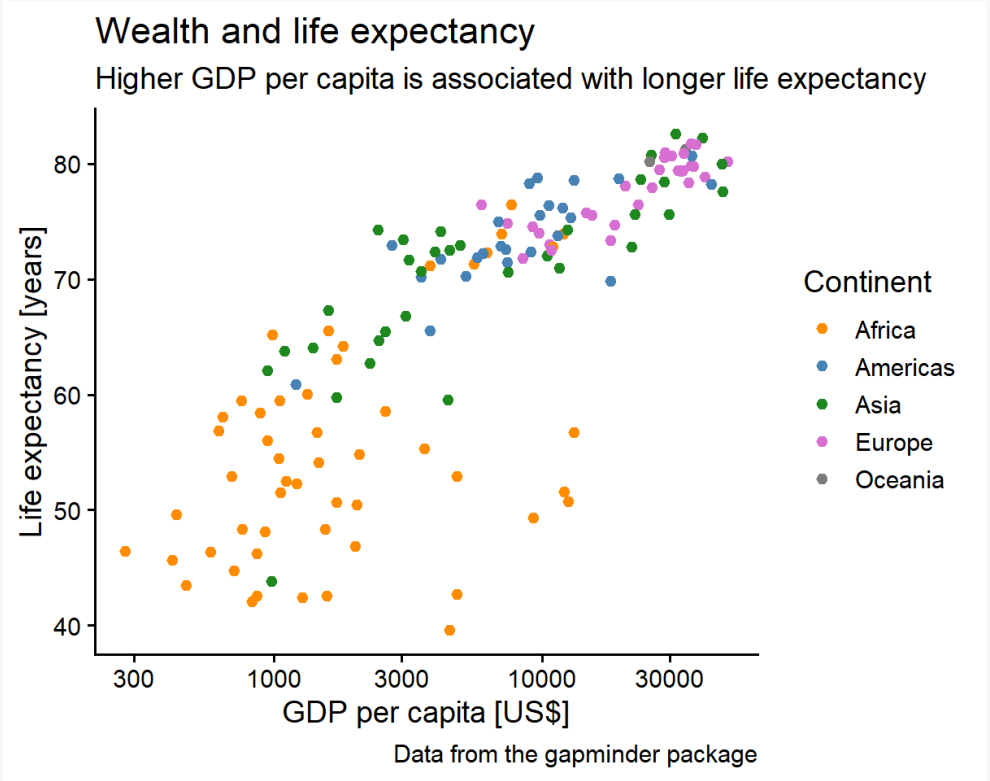
`labs`: Change axis and legend titles and add plot title



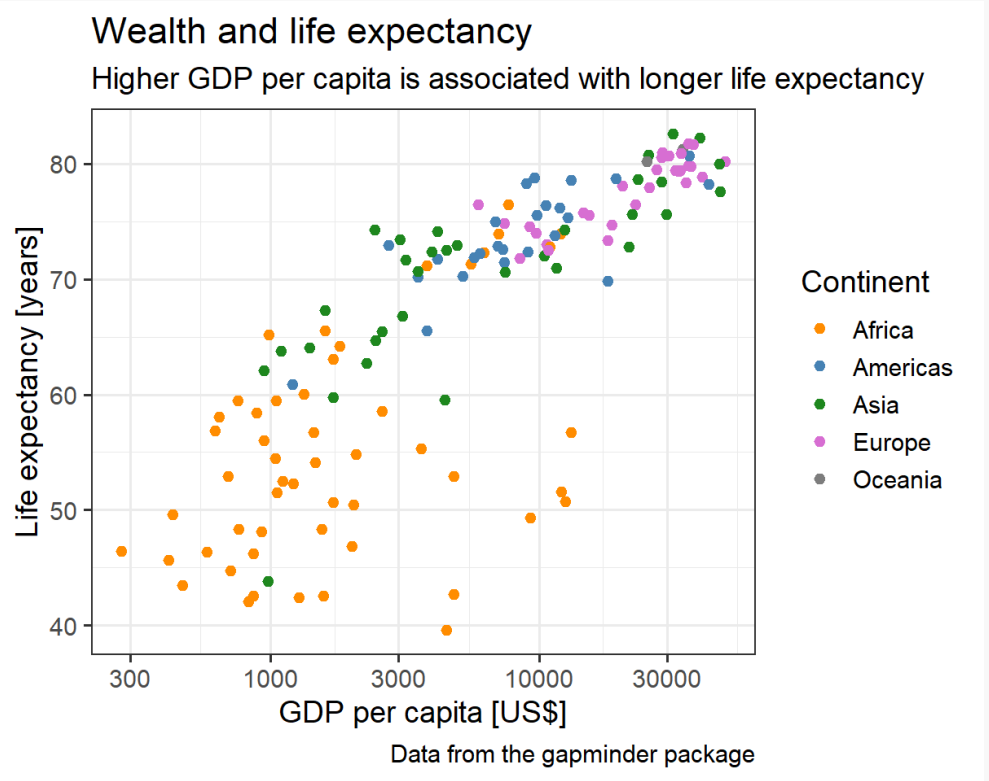
`theme_*`: change appearance

`ggplot2` offers many pre-defined themes that we can apply to change the appearance of a plot.

```
my_plot +
  theme_classic()
```



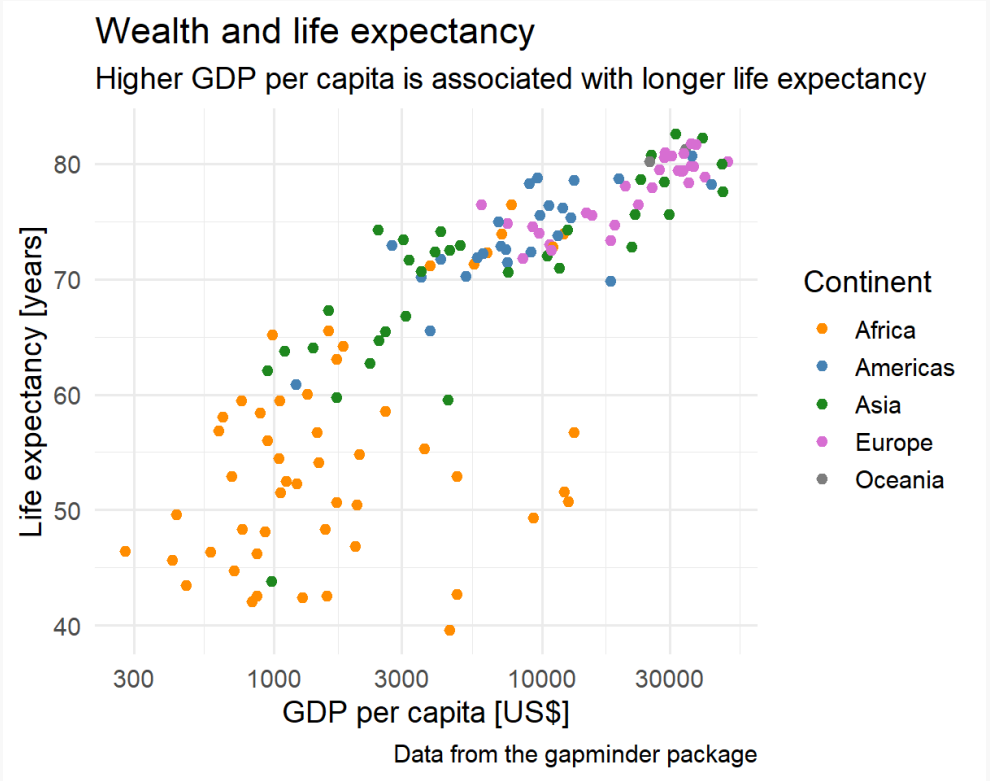

```
my_plot +
  theme_bw()
```



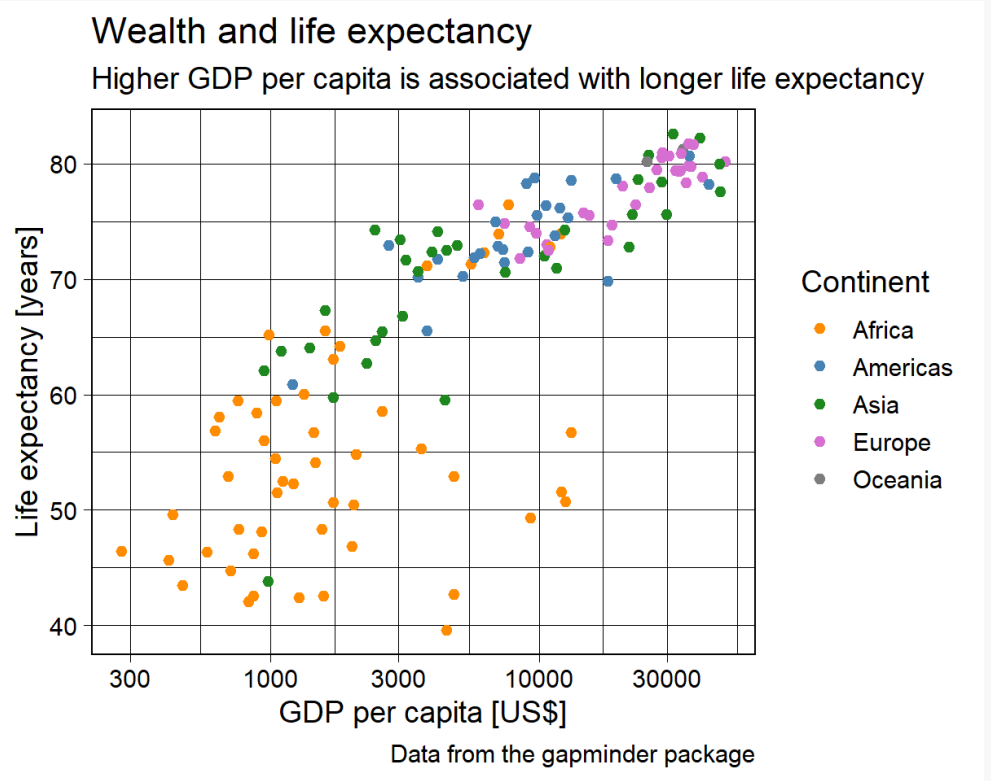
`theme_*`: change appearance

`ggplot2` offers many pre-defined themes that we can apply to change the appearance of a plot.

```
my_plot +
  theme_minimal()
```




```
my_plot +
  theme_linedraw()
```

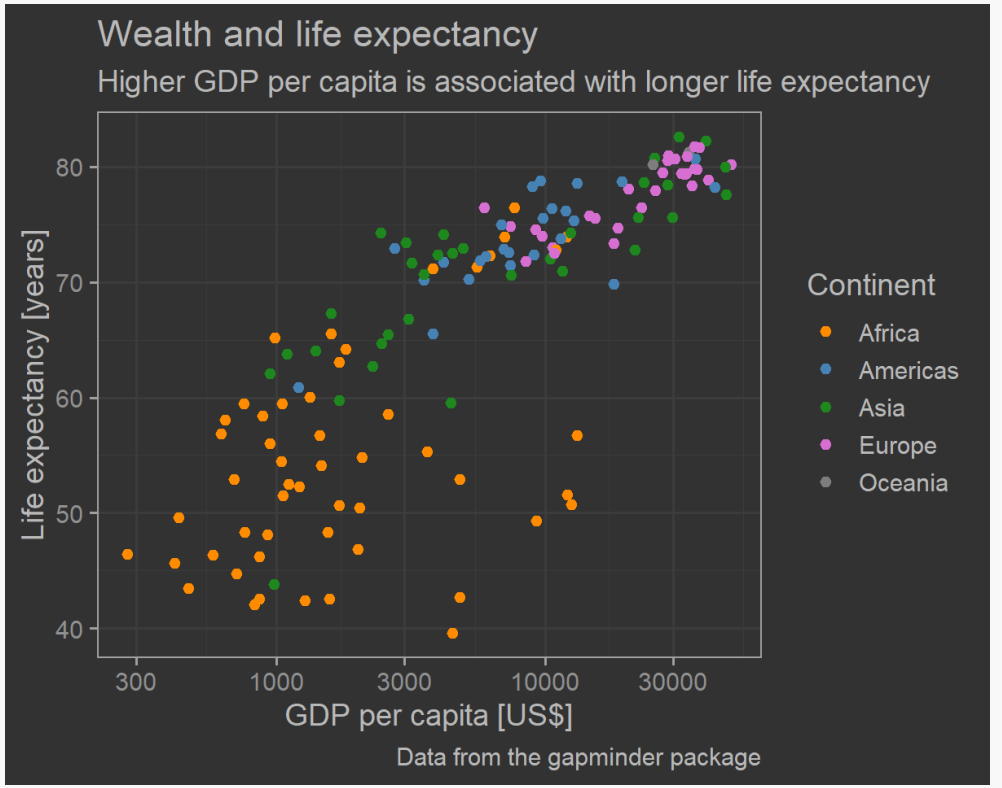


`theme_*`: change appearance

Since `ggplot2` v. 4.0.0, you can change overall color choices of a pre-defined theme.

```
my_plot +
  theme_bw(
    ink = "#BBBBBB",
    paper = "#333333"
  )
```

- `paper`: affects background elements
- `ink`: affects foreground elements (text, lines, points, ...)
- `accent`: affects elements that are used to highlight information (like `geom_smooth()` lines)

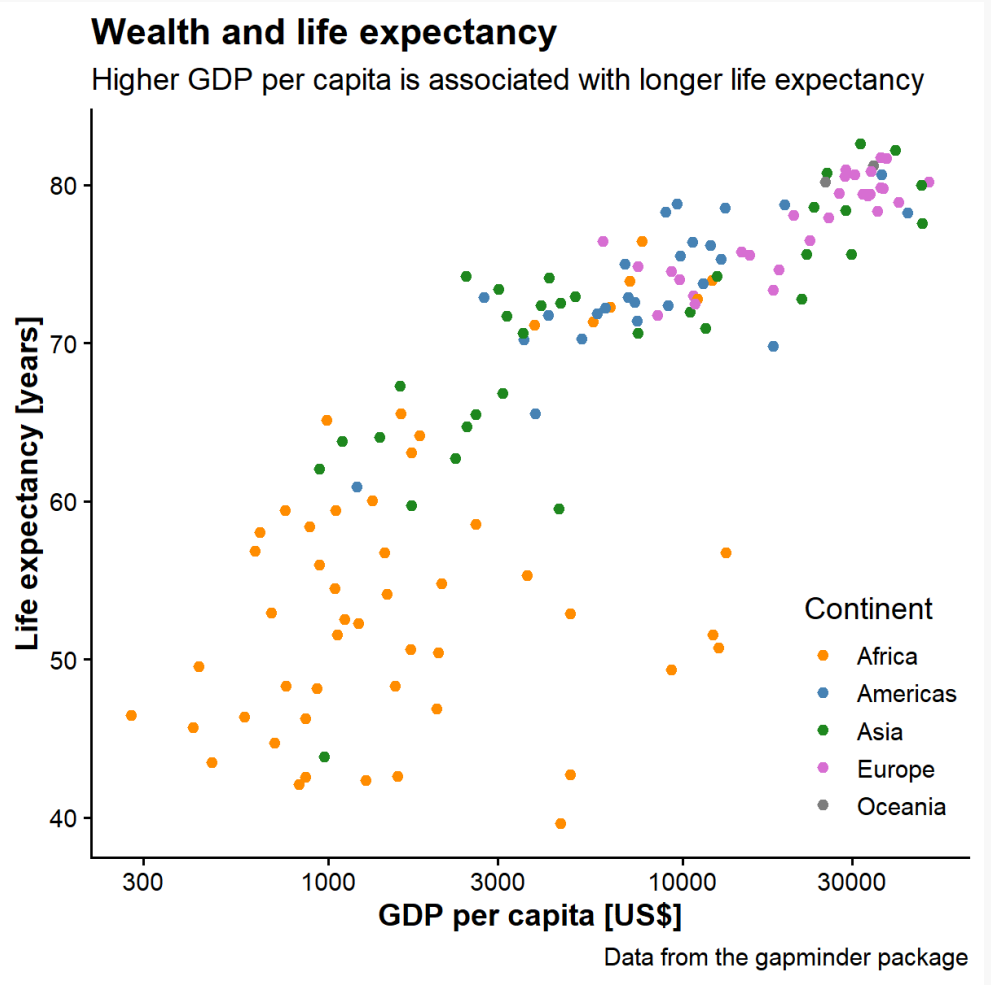


`theme()`: fine-tune individual elements

```
my_plot <- my_plot +
  theme_classic() +
  theme(
    # Move legend inside the plot
    legend.position = "inside",
    # Define the position with x and y coordinates (0-1)
    legend.position.inside = c(0.90, 0.2),
    # Make the title bold
    plot.title = element_text(face = "bold"),
    # Make axis titles larger and bold
    axis.title = element_text(size = 14, face = "bold")
  )
my_plot
```

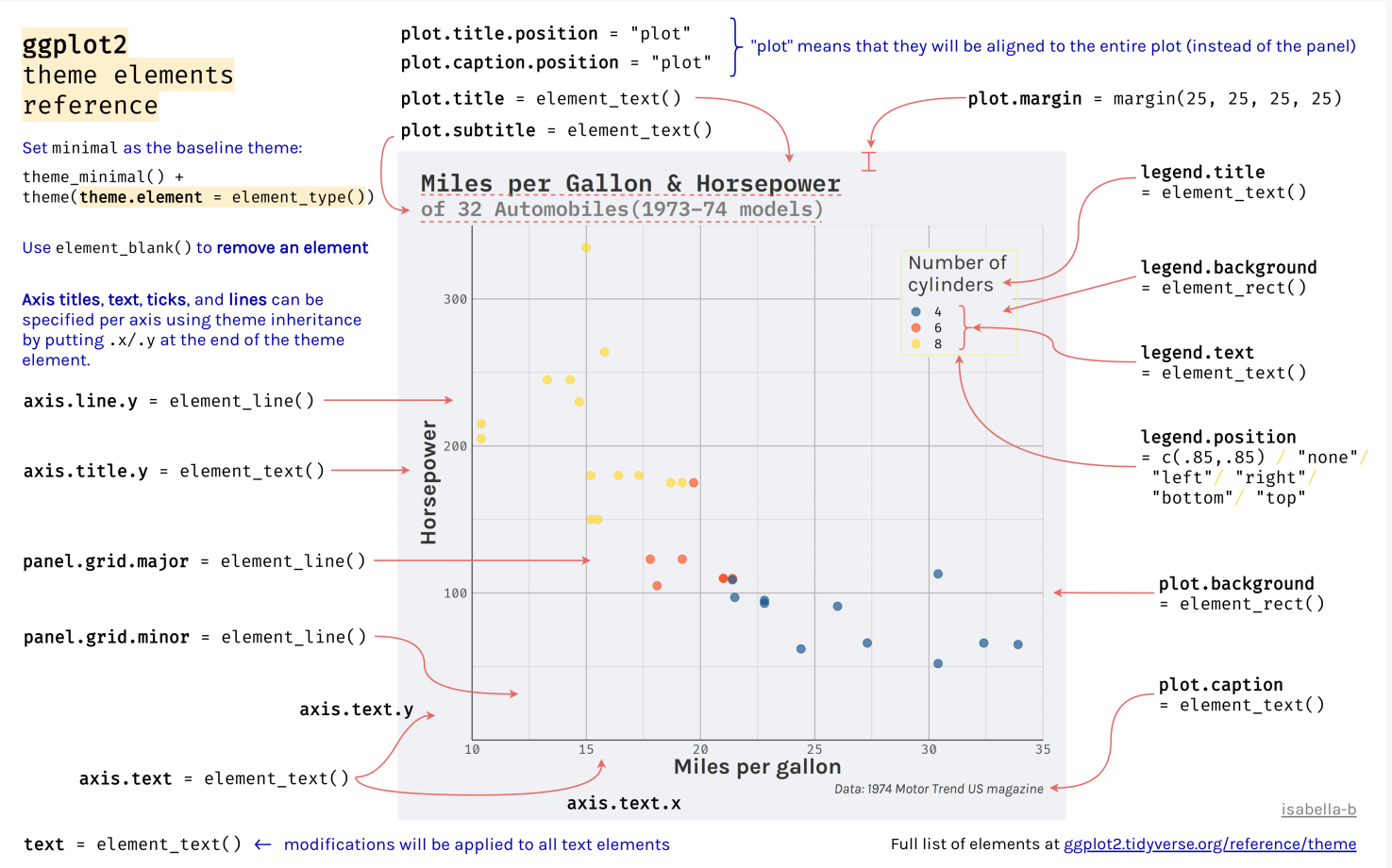
The basic functioning of theme elements is:

```
theme(
  element_name = element_function()
)
```



`theme()`: fine-tune individual elements

- Check `?theme` for all theme elements and options
- Search: "ggplot theme ..." (e.my_plot. "ggplot theme remove legend")



Theme reference sheet by Isabella Benabaye

ggsave()

A ggplot object can be saved on disk in different formats.

Without specifications:

```
# save plot my_plot in img as my_plot.pdf
ggsave(filename = "img/my_plot.pdf", plot = my_plot)
# save plot my_plot in img as my_plot.png
ggsave(filename = "img/my_plot.png", plot = my_plot)
```

Or with specifications:

```
# save a plot named my_plot in the img directory under the name my_plot.png
# with width 16 cm and height 9 cm
ggsave(
  filename = "img/my_plot.png",
  plot = my_plot,
  width = 16,
  heigth = 9,
  units = "cm"
)
```

Have a look at `?ggsave` to see all options.

Now you

Task 2 (30 min)

Make your penguin plots more beautiful

Find the task description here